Statement-level bug localization (ISBL): complete project documentation

Reproduction of Li, Yue, Lan, Cui, IEEE SMC 2023, on Defects4J Lang-1, Lang-2 and Lang-3 (Windows, no WSL)
Compiled 2026-10-27

#	Document	Source file	Page
1	**REPORT.md** Illustrated report: pipelines, metrics with formulas, Lang-1 and Lang-2 results	`docs/REPORT.md`	2
2	**experiments.md** Experiments on Lang-1, Lang-2, Lang-3: IR models, SFL formulas, hybrid ranking	`docs/experiments.md`	15
3	**literature.md** Literature scan for metrics and methods	`docs/literature.md`	19
4	**reproduction.md** Reproduction status and answers to the plan's questions	`docs/reproduction.md`	21
5	**baseline.md** Baseline metadata: pinned versions and commands	`docs/baseline.md`	23
6	**environment-audit.md** Environment audit and blockers	`docs/environment-audit.md`	24
7	**README.md** Repository README: layout, setup, how to run	`README.md`	25

All documents of the repository are combined in this order. Status: the pipeline works end to end and is tested, the paper's own results are **not** reproduced (3 of 373 bugs run). docs/architecture.md is empty and not included.

Finding the buggy line automatically: what we built, what we changed, and what we found

Project: reproduction and improvement of "*Statement-level Software Bug Localization based on Information Retrieval and Spectrum*" (ISBL; Li, Yue, Lan, Cui; IEEE SMC 2023)
Data: Defects4J v1.4.0, Apache Commons Lang bugs **Lang-1** and **Lang-2** (Lang-3 is summarised in `docs/experiments.md`)
Date of results: 2026-10-27 | **Platform:** Windows 11, JDK 8, Python 3.7 and 3.12, no WSL

> **Honest status.** The pipeline works end to end on real bugs and is tested (104 unit tests). The paper's headline numbers are **not** reproduced, because that needs all 373 bugs and we ran three (this report details two). Differences between methods below are small counts on very few bugs, so treat them as indications, not proof. No parameter was tuned on these bugs.

1. One-page summary

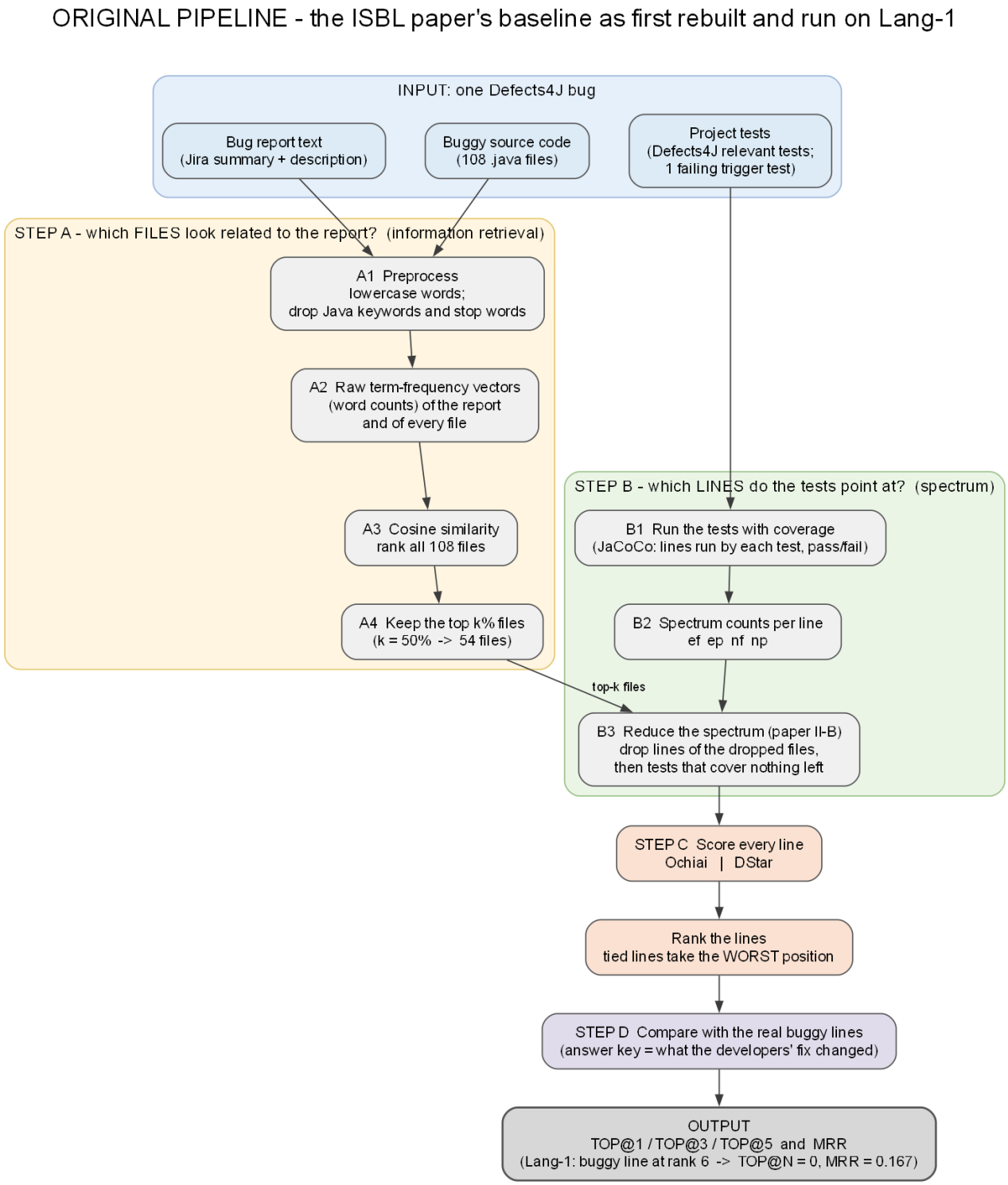
- **The problem.** A program has thousands of lines and one of them is wrong. Finding it by hand is slow, so researchers rank lines by "how suspicious" they are.
- **The paper's idea (ISBL).** Use two clues. *Clue 1:* the **bug report** (text written by a user) tells us which **files** look related. *Clue 2:* the **tests** (which lines each test ran, and which test failed) tell us which **lines** are suspicious. Filter the files first, then rank only the lines inside the likely files.
- **Step 1 of our work (the original pipeline).** We rebuilt that method from the paper and ran it on Lang-1: it works, but the buggy line is ranked 6th because six lines tie.
- **Step 2 (after faculty review).** We were asked to add more metrics, check the cosine similarity properly, try TF-IDF and better IR measures, read the literature, and run other Lang bugs. We did all of that, on Lang-1, Lang-2 and Lang-3.
- **What we found**
  1. The cosine arithmetic is correct (it matches scikit-learn and SciPy to 1e-16). The weak point is the **weighting**: with raw word counts, ordinary words ("not", "string", "java") decide the ranking. TF-IDF-style weighting fixes this: the main variants (TF-IDF, rVSM) rank the buggy file **1st** on Lang-1 and Lang-2, where the paper's raw-count cosine ranks it **8th** and **23rd**.
  2. A better file ranking lets the filter be **more aggressive without losing the bug**: at k = 10% the paper's IR throws the buggy file away on Lang-2 (the bug can then never be found), while TF-IDF and rVSM keep it.
  3. Nine other spectrum formulas give the **same** ranking as Ochiai: lines run by exactly the same tests have exactly the same counts, and no formula can tell them apart.
  4. Using the bug report's words to break those exact ties moved the buggy line on Lang-1 from "somewhere between 1st and 6th" (average 3.5) to **2nd**; it changed nothing on Lang-2.
  5. Almost all run time (98%) was one slow step, collecting coverage; we made it about **8× faster** with identical output.

2. Vocabulary in plain words

Term	Meaning
Bug report	The written complaint (Jira title + description), e.g. "*NumberUtils does not handle Long Hex numbers*"
Statement / line	One executable line of Java code; what we want to rank
Test	A small program that checks one behaviour; it **passes** or **fails**. Each bug has at least one failing test
Coverage / spectrum	For every test, the set of lines it ran. Together with pass/fail this is the *spectrum*
IR (information retrieval)	Ranking files by how similar their words are to the words of the bug report

Term	Meaning
SFL (spectrum-based fault localisation)	Ranking lines by suspicion computed from the spectrum with a formula (Ochiai, DStar, ...)
k	The share of top-ranked files kept as "suspicious" (k = 50% of 108 files = 54 files)
Tie	Several lines with exactly the same score; their true order is unknown
Answer key	The lines the developers' real fix changed; used **only to grade** the result, never by the method

3. The original pipeline



(Drawn by us from the paper's description and our first implementation; the paper's own Figure 1 is not reproduced here.)

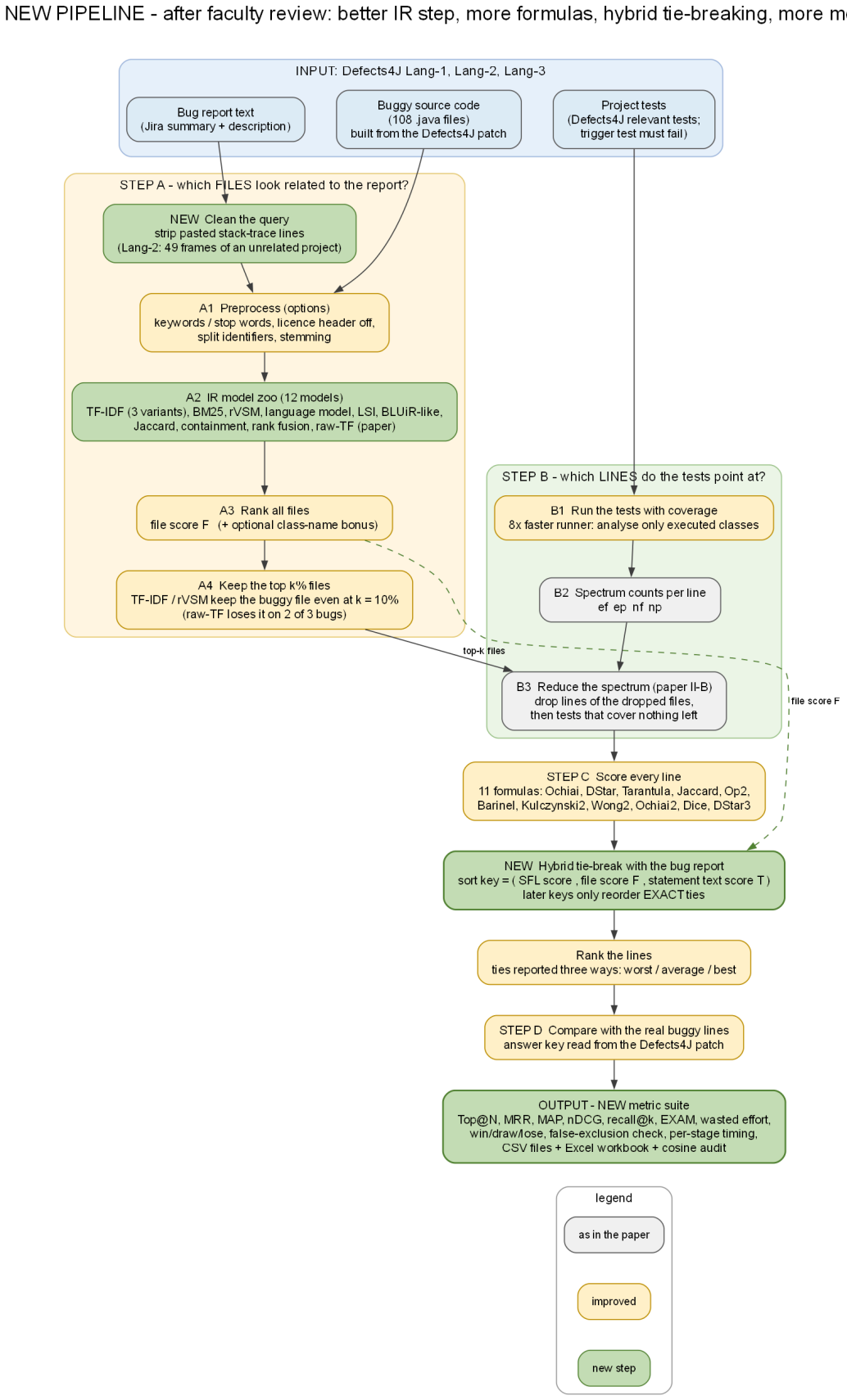
Stage	What it does	Where (repo)
Input	Bug report, buggy source (108 files), project tests	`data/bug_reports/`, `work-lang/`
A1-A3	Lowercase words, drop Java keywords and stop words, count words, cosine similarity between report and every file	`isbl/baseline/ir.py`
A4	Keep the top k% of files	`ir.select_top_k`
B1-B2	Run tests under JaCoCo, record lines per test, build `ef ep nf np` per line	`isbl/baseline/java/CoverageRunner.java`, `spectrum.py`
B3	Paper section II-B: drop lines of dropped files, then tests that cover nothing left	`spectrum.reduce_tests`
C	Ochiai or DStar per line; rank lines; ties counted at the **worst** position	`sfl.py`, `ranking.py`
D	Compare with the answer key: TOP@1/3/5, MRR	`isbl/evaluation/metrics.py`

First result (Lang-1, k = 50%): the buggy file was ranked 8th of 108 (still kept), six lines tied at the top score (three of them buggy), so the buggy line sits at rank 6 with worst-case ties: TOP@1/3/5 = 0/0/0, MRR = 0.167. ISBL (with filtering) and plain Ochiai/DStar gave the same ranking.

4. What the faculty review asked, and what we did

Request	Done
More metrics for comparison	a file-level and a statement-level metric suite (section 6)
Check the cosine similarity properly	`scripts/run/cosine_audit.py` (section 9)
Try TF-IDF and more proper IR measures	12 models in `isbl/baseline/ir_models.py` (section 6.3)
Find relevant papers and add their metrics	`docs/literature.md`: 11 sources (abstracts and snippets only, not full papers)
Run on other Lang bugs	Lang-2 and Lang-3 prepared with `scripts/bugs/prepare_lang_bug.py`
Find more types of bugs	Lang-1 = wrong condition; Lang-2 and Lang-3 = *omission* bugs (a needed check is missing)

5. The new pipeline



Part	Original	New
Query	whole report as written	**stack-trace lines removed** (Lang-2's report holds 49 frames from an unrelated project)

Part	Original	New
Word preparation	keywords + stop words removed	plus options: licence header off, identifiers split (`createNumber` -> create, number), stemming
File ranking	raw word-count cosine	**12 IR models** (TF-IDF, BM25, rVSM, language model, LSI, BLUiR-like, rank fusion, ...) and an optional class-name bonus
Filter	k = 50%	same mechanism, but the better IR keeps the buggy file at k = 10%
Coverage	analysed every class after every test	analyses **only executed classes**: about 8× faster, identical output
Line scoring	Ochiai, DStar	**11 formulas**
Ties	counted at the worst position	reported **worst / average / best**; **hybrid tie-break** with bug-report words
Metrics	TOP@1/3/5, MRR	full suite (below) + false-exclusion check + per-stage timing + CSV/Excel outputs
Bugs	Lang-1	Lang-1, Lang-2, Lang-3, answer key read straight from the Defects4J patch

6. The metrics, with formulas and small examples

6.1 The spectrum counts and the line-scoring formulas

For **one line** we count tests:

Symbol	Meaning
`ef`	failing tests that **run** the line
`ep`	passing tests that run the line
`nf`	failing tests that do **not** run the line
`np`	passing tests that do not run the line

Toy example. Four tests (T1 fails) and three lines; a 1 means "the test ran the line".

test	result	line 1	line 2	line 3
T1	**FAIL**	1	1	0
T2	pass	1	0	1
T3	pass	1	0	0
T4	pass	0	0	1
(ef, ep, nf, np)		(1, 2, 0, 1)	(1, 0, 0, 3)	(0, 2, 1, 1)

The idea: a line run by the failing test and by few passing tests is suspicious.

```
Ochiai   = ef / sqrt( (ef + nf) * (ef + ep) )          line 1: 1/sqrt(1*3) = 0.577    line 2: 1/sqrt(1*1) =
1.0    line 3: 0
DStar(2) = ef^2 / (ep + nf)                            line 1: 1/2 = 0.5              line 2: 1/0 -> infinity
(top)  line 3: 0
Tarantula = (ef/(ef+nf)) / ( ef/(ef+nf) + ep/(ep+np) )  line 1: 1/(1+2/3) = 0.6        line 2: 1.0
line 3: 0
```

Line 2 (only the failing test runs it) is ranked first by all three. The other formulas we added use the same four counts:

Formula	Expression	Value for the real Lang-1 tie lines (1, 3, 0, 166)
Ochiai	`ef / sqrt((ef+nf)(ef+ep))`	0.500
DStar (2) / DStar (3)	`ef^2/(ep+nf)` / `ef^3/(ep+nf)`	0.333 / 0.333
Tarantula	see above	0.983
Jaccard	`ef / (ef+nf+ep)`	0.250
Sorensen-Dice	`2ef / (2ef+nf+ep)`	0.400
Kulczynski2	`0.5 * (ef/(ef+nf) + ef/(ef+ep))`	0.625
Op2	`ef - ep/(ep+np+1)`	0.982
Barinel	`1 - ep/(ep+ef)`	0.250
Wong2	`ef - ep`	-2

Formula	Expression	Value for the real Lang-1 tie lines (1, 3, 0, 166)
Ochiai2	`ef*np / sqrt((ef+ep)(nf+np)(ef+nf)(ep+np))`	0.496

All of them depend only on `(ef, ep, nf, np)`. **Two lines with the same four numbers get the same score from every formula**; that is why switching formulas could not fix the ties below.

6.2 Ties and how ranks are counted

Six lines share the top score. They occupy positions 1 to 6, but nobody knows their order. Three ways to count:

Rule	Rank given to each of the six	Meaning
worst	6	the bug is the last of the tied lines
average	3.5	expected position
best	1	the bug is the first of the tied lines

The paper does not say which rule it uses, so we report all three. Results below use *worst* for TOP@N (the cautious choice) and *average* for MRR/MAP/nDCG/EXAM unless the column says otherwise (`w/a/b`).

6.3 IR models for ranking files (Step A)

Each file and the report are turned into "bags of words". A score says how similar they are.

Cosine similarity (what the paper uses, on raw word counts):

```
cos(q, d) = sum( q_t * d_t ) / ( sqrt(sum q_t^2) * sqrt(sum d_t^2) )        value between 0 and 1
```

Example: report `q = (hex 1, common 1)`; file A is "common" written 10 times, file B is "hex" once. Cosine A = 10 / (1.414 x 10) = 0.707, cosine B = 1 / (1.414 x 1) = 0.707 : **a tie**, although B contains the rare, informative word and A only repeats an ordinary one.

TF-IDF weights each word by how rare it is across files, so ordinary words count less:

```
idf(t)    = ln( (1 + N) / (1 + df(t)) ) + 1          N = number of files, df = files containing the word
weight    = (1 + ln tf) * idf(t)                      "sublinear" tf; then cosine as above
```

In the example (N = 3): idf(common) = 1.288 (in 2 files), idf(hex) = 1.693 (in 1 file). TF-IDF cosine: A = **0.605**, B = **0.796**: now B wins. (The authors' script uses `idf = ln(N/df) + 1` without smoothing; we kept that as `tfidf_authors`.)

BM25 (Okapi) also rewards rare words but limits the credit for repeating a word and corrects for file length:

```
BM25(q,d) = sum over query words t of  idf(t) * tf*(k1+1) / ( tf + k1*(1 - b + b*|d|/avgdl) ),   k1 = 1.2, b =
0.75
idf(t)    = ln( 1 + (N - df + 0.5) / (df + 0.5) )
```

Example: N = 100, df = 10, tf = 3, file of average length: idf = 2.264, tf-part = 3x2.2/(3+1.2) = 1.571, score = **3.557**. The same tf = 3 in a file twice as long: **2.930**.

rVSM (BugLocator): `rVSM = g(length) * cosine` with log-tf and `idf = ln(N/df)`, and `g = 1/(1+e^(-N(len)))` where `N(len)` scales file length to 0..1: g = 0.5 for the shortest file and 0.731 for the longest (a mild bonus for big files).

Dirichlet language model: `score = sum over query words ln( (tf + mu*p(t)) / (|d| + mu) )`, mu = 2000, `p(t)` = share of the word in all files. Example: tf = 2, |d| = 100, p = 0.01 -> ln(22/2100) = -4.559 (less negative = better).

Jaccard `|Q and D| / |Q or D|` and **containment** `|Q and D| / |Q|`: example Q = {a, b}, D = {a, b, c}: 0.667 and 1.0. **LSI** projects TF-IDF vectors onto 50 latent directions (SVD) and compares there. **BLUiR-like** computes BM25 separately between {title, description} and {class name, method names, identifiers, comments} of each file and adds the scores. **Rank fusion (RRF)** `score = sum 1/(60 + rank_i)`: a file ranked 4th by two models (0.03125) beats one ranked 1st by one and 10th by another (0.03068).

Preparation options. *split identifiers*: `createNumber` -> `createnumber`, `create`, `number`. *stem*: `numbers` -> `number`. *clean*: delete stack-frame lines (`at pkg.Class.method(File.java:12)`) from the report. *name bonus*: add to a file's score if its class name appears as a word in the report.

Choosing the files. `kept = ceil(k * N)` files: k = 10% of 108 files is **11 files**, k = 50% is **54**.

IR diagnostics we log.

```
percentile = (rank of the buggy file) / N         the smallest k that would keep it   (Lang-1: 8/108 = 0.074)
gap        = cosine(rank 1) - cosine(rank 2)      small gap = the model is not decisive
entropy    = - sum p_i log2 p_i,   p_i = score_i / sum(scores)      max for 108 equal scores = 6.75 bits; low =
confident
```

6.4 Evaluation metrics

File level (did the IR step find the right file?): the buggy file's rank `r` among N files.

Metric	Formula	Small example
Top@N	1 if `r <= N`, else 0 (summed over bugs)	ranks (1, 4, 12): Top@1 = 1, Top@3 = 1, Top@5 = 2
MRR	mean of `1/r`	ranks 1, 3, 5: (1 + 1/3 + 1/5)/3 = **0.511**
AP / MAP	mean over buggy items of `i / rank_i` (÷ number of buggy items; one never found counts 0)	buggy at ranks 2 and 4: (1/2 + 2/4)/2 = **0.5**; with a third one never found: 1.0/3 = **0.333**
nDCG@k	`sum 1/log2(rank+1)` for buggy items with rank <= k, divided by the best possible	one buggy item at rank 3: 1/log2(4) = **0.5**
recall@k% of files	share of buggy files inside the top k% of files	buggy file rank 23 of 108, k = 10%: top 11 -> **0** (lost)
precision@k	buggy items in the top k, divided by k	ranks {1, 3, 9}, k = 5: 2/5 = **0.4**

Statement level (did we find the right line?): rank of the **best-ranked** buggy line.

Metric	Formula	Example
TOP@N	number of bugs whose best buggy line has rank `<= N`	Lang-1 worst-case rank 6: TOP@1/3/5 = 0/0/0
MRR	mean of `1/rank`	rank 6 -> 0.167; average rank 3.5 -> 0.286
MAP, nDCG@10	as above, over **all** buggy lines (Lang-1 has three)	
recall@10	buggy lines in the top 10 / all buggy lines	ranks {1, 3, 9} of 4 buggy lines: 3/4
EXAM	`rank / total statements` : share of the list a developer reads	rank 6 of 981 = **0.0061** (worst) or 3.5/981 = **0.0036** (average)
wasted effort	`rank - 1` : lines read before the bug	5 (worst) or 2.5 (average)
win / draw / lose	per bug, compare two methods' best rank	A = (2, 5, none), B = (3, 5, 4): 1 win, 1 draw, 1 loss
bootstrap CI, Wilcoxon p	uncertainty of the mean / paired test	reported for completeness; with 2-3 bugs they say nothing

False exclusion. If the buggy file is not among the kept files, its lines are not in the reduced spectrum, so the bug can **never** be found. We count, per setting, how many bugs lose their buggy file ("lost") and how much of the spectrum is removed ("cut"). This is the failure the paper itself describes for Math-51.

6.5 The hybrid tie-break (our extension)

Lines with identical `(ef, ep, nf, np)` cannot be separated by the tests. The bug report can: lines that mention the report's words are more likely to be the bug.

```
T(line) = sum of idf(t)  over the words t shared by the line's source text and the bug report
F(file) = the IR score of the line's file, scaled to 0..1
sort key = ( SFL score , F , T )     compared left to right: F and T only reorder EXACT ties of the SFL score
```

Real example, the six tied lines of Lang-1 (all have Ochiai 0.5, same file so same F):

Line	Source text	Buggy?	T	Rank (worst-case)
474	`return createInteger(str);`	no	7.007	1
467	`final int hexDigits = str.length() - pfxLen;`	**yes**	5.979	**2**
468	`if (hexDigits > 16) { // too many for Long`	**yes**	4.771	4 (tied with 471)
471	`if (hexDigits > 8) { // too many for an int`	**yes**	4.771	4
462	`pfxLen += pfx.length();`	no	1.208	5
463	`break;`	no	0.000	6

The best buggy line moves from "rank 1 to 6 (average 3.5)" to rank **2**. It is not perfect: line 474 is not buggy but shares the word `createInteger` with the report, so it comes first. A plain **linear blend** (`0.4*SFL + 0.3*F + 0.3*T`, each scaled to 0..1) was also tried; it can overrule the tests, and on Lang-2 it made things worse (below).

---

7. Results: Lang-1 (LANG-747, a wrong-condition bug)

The bug. `NumberUtils.createNumber("0x80000000")` crashes: the code decides int vs long by counting hex digits instead of checking the value. **Answer key:** `NumberUtils.java` lines 467, 468, 471. **Setup:** 108 files, 173 tests (1 failing: `TestLang747`), 981 statements in the spectrum, report of 662 characters.

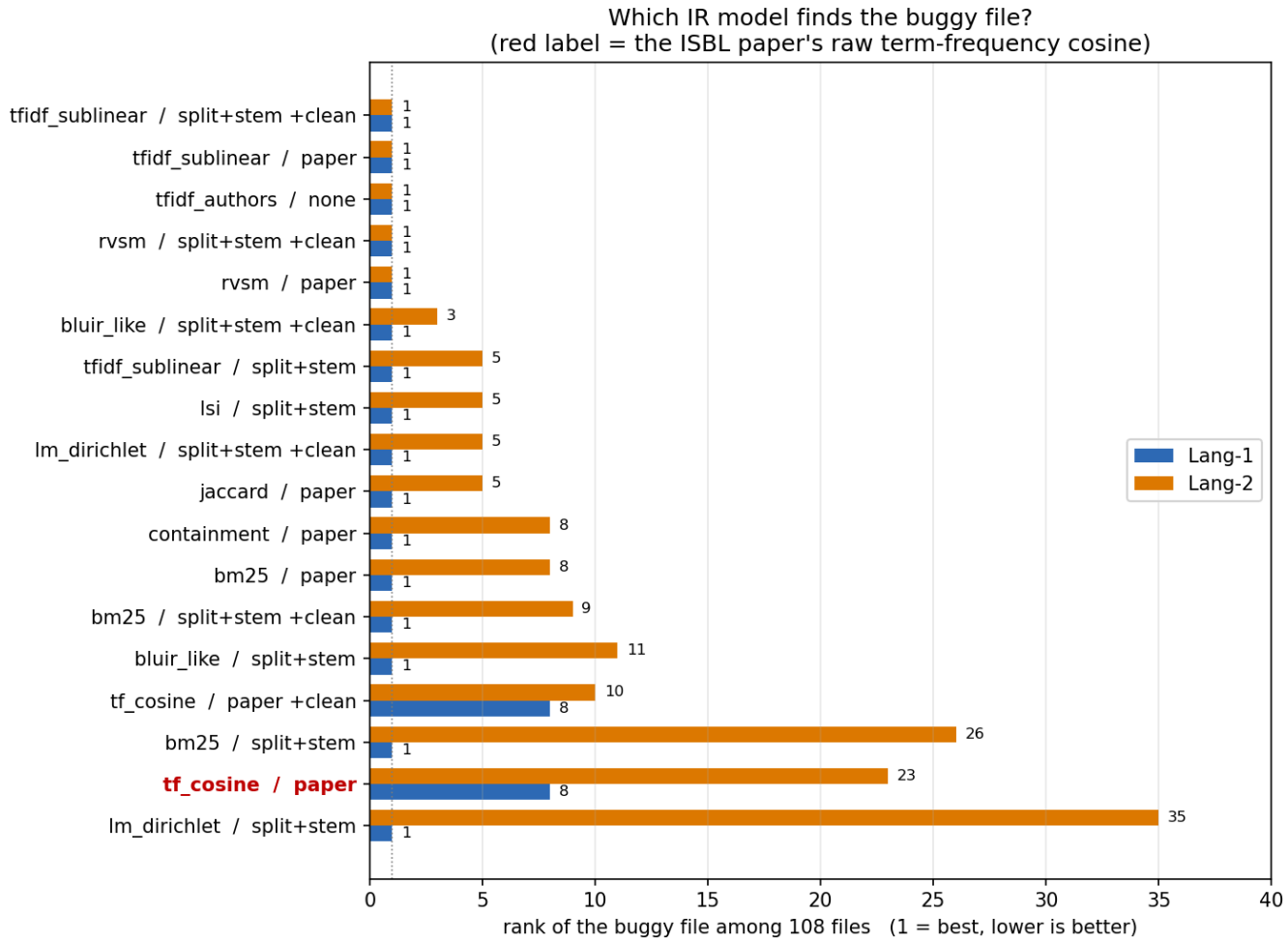
Standard pipeline, k = 50% (paper-style IR).

Method	Tests	Statements	IR rank of buggy file	Tie group at the buggy line	Rank w / a / b	TOP@1/3/5 (w)	MRR (w)
Ochiai	173	981	8	6 lines (3 buggy)	6 / 3.5 / 1	0 / 0 / 0	0.167
ISBL + Ochiai	170	913	8	6 lines (3 buggy)	6 / 3.5 / 1	0 / 0 / 0	0.167
DStar, ISBL + DStar	same as Ochiai						

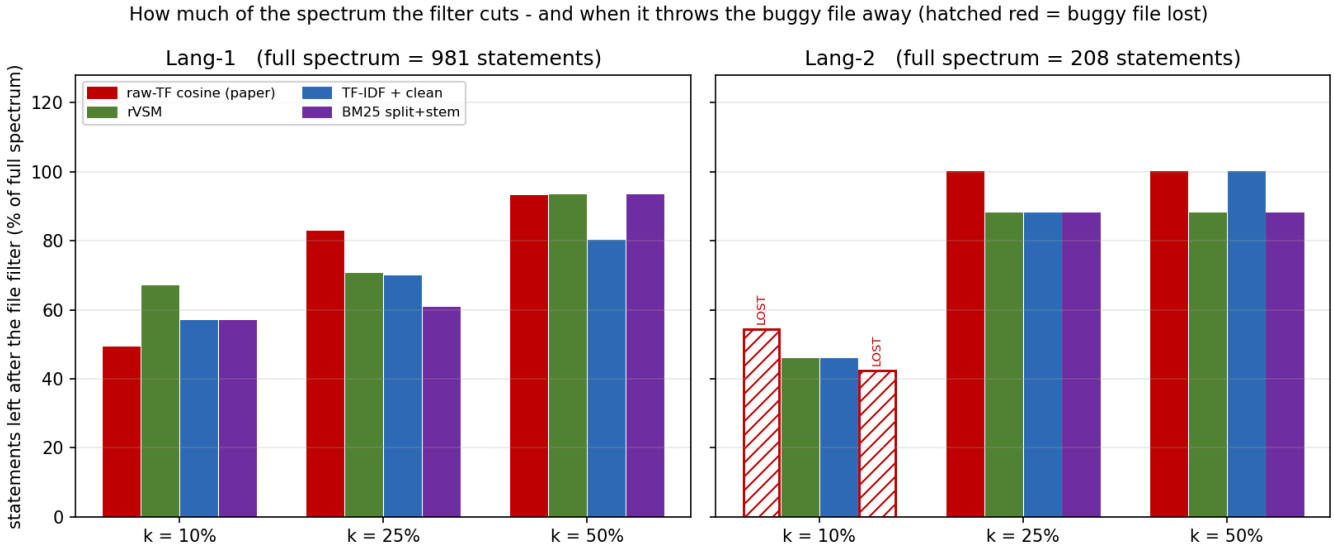
IR facts for the buggy file: cosine 0.1198 against 0.1560 for the top file (`Validate.java`), gap 0.0024, entropy 6.65 of at most 6.75 bits, matrix density 1.4% (only 1.4% of test-line cells are 1).

File level: rank of the buggy file (Lang-1 | Lang-2).

IR configuration	Lang-1	Lang-2
raw-TF cosine (the paper)	**8**	**23**
raw-TF cosine + clean query	8	10
TF-IDF, authors' script (no preprocessing)	1	1
TF-IDF sublinear / paper preprocessing	1	1
TF-IDF sublinear / split + stem	1	5
TF-IDF sublinear / split + stem + clean	1	1
rVSM / paper	1	1
rVSM / split + stem	1	2
rVSM / split + stem + clean	1	1
BM25 / paper	1	8
BM25 / split + stem	1	26
BM25 / split + stem + clean	1	9
Dirichlet language model / split + stem	1	35
Dirichlet language model / split + stem + clean	1	5
LSI / split + stem	1	5
BLUiR-like / split + stem	1	11
BLUiR-like / split + stem + clean	1	3
Jaccard / containment	1 / 1	5 / 8
rank fusion (BM25, TF-IDF, rVSM)	1	6
TF-IDF or BM25, split + stem, + class-name bonus	1	1



Filtering at several k: Lang-1 (bars show the share of the 981 statements that remain; the buggy file was never lost on Lang-1).

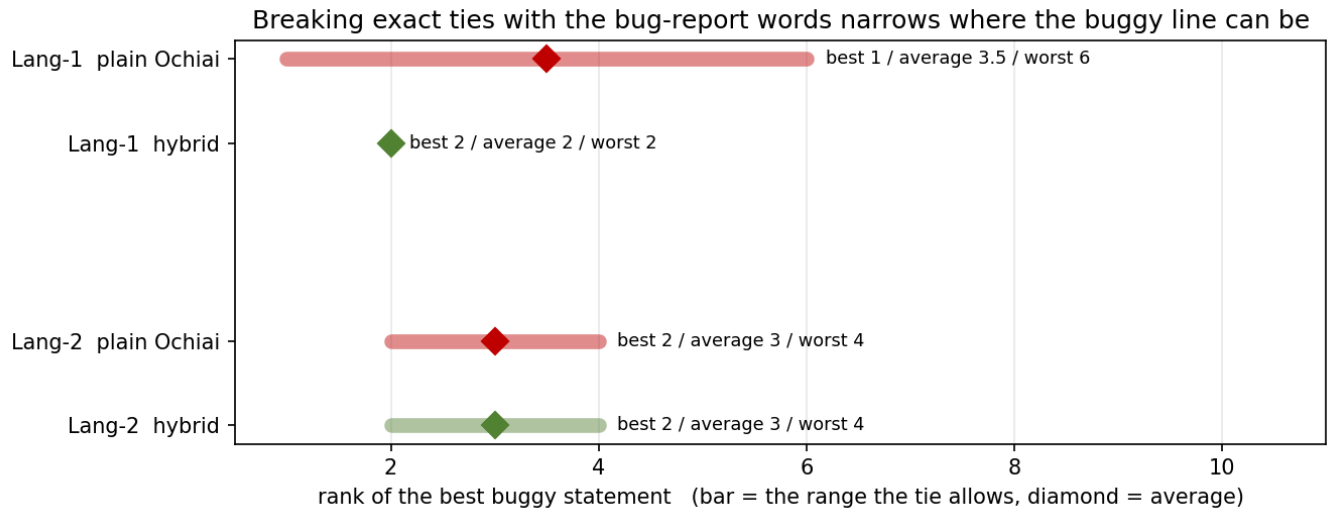


IR model	k = 50%	k = 25%	k = 10%
raw-TF cosine (paper)	913 (93%)	811 (83%)	481 (49%)
rVSM / paper	914	692	656
TF-IDF sublinear / split+stem + clean	786 (80%)	684 (70%)	558 (57%)
BM25 / split+stem	914	594	558

Statement level: Lang-1 (`w` = worst-case ties, `a` = ties averaged; every row keeps all three buggy lines in the spectrum).

Method	Stmts	Tests	Best buggy rank w / a / b	TOP@1/3/5 (w)	MRR w	MRR a	MAP a	nDCG@10 a	EXAM a	Wasted a
SFL: Ochiai (= DStar, Tarantula, Jaccard, Op2, Barinel, Kulczynski2, Ochiai2, Dice, DStar3)	981	173	6 / 3.5 / 1	0/0/0	0.167	0.286	0.571	0.649	0.0036	2.5
SFL: Wong2	981	173	716 / 628 / 540	0/0/0	0.001	0.002	0.003	0.000	0.6402	627
ISBL, raw-TF (paper), k = 50%	913	170	6 / 3.5 / 1	0/0/0	0.167	0.286	0.571	0.649	0.0036	2.5
ISBL, TF-IDF + clean, k = 50%	786	168	6 / 3.5 / 1	0/0/0	0.167	0.286	0.571	0.649	0.0036	2.5
ISBL, TF-IDF + clean, k = 10%	558	150	6 / 3.5 / 1	0/0/0	0.167	0.286	0.571	0.649	0.0036	2.5
Hybrid, file score F only (`tie_file`)	786	168	6 / 3.5 / 1	0/0/0	0.167	0.286	0.571	0.649	0.0036	2.5
Hybrid, statement text (`tie_text`)	786	168	**2 / 2 / 2**	**0/1/1**	**0.500**	**0.500**	**0.643**	**0.729**	**0.0020**	**1.0**
Hybrid, linear blend (a = 0.3, b = 0.3)	786	168	2 / 2 / 2	0/1/1	0.500	0.500	0.643	0.729	0.0020	1.0

Reading Lang-1.
1. The paper's IR ranks the buggy file 8th; every idf-weighted model ranks it 1st.
2. No formula, and no amount of filtering, changes the statement ranking: the six tied lines are in the buggy file, and exactly the same four tests (one failing, three passing) run all six.
3. Filtering with a better IR still pays off: it removes 20% of the statements at k = 50% and 43% at k = 10% (the paper's IR: 7% and 51% - and on Lang-2 the latter throws the bug away, see below), with the ranking unchanged.
4. Breaking the exact ties with the report's words moves the best buggy line from 1-to-6 (average 3.5) to 2nd, and TOP@3 from 0 to 1.



8. Results: Lang-2 (LANG-879, an omission bug)

The bug. `LocaleUtils.toLocale(...)` does not reject a locale string containing `#` (new in Java 7, e.g. `ja_JP_JP_#u-ca-japanese`). The fix **adds** a check; the buggy version is **missing** it. The answer key for an omission is the first line after the gap: **line 92** (`final int len = str.length();`). **Setup:** 108 files, only 14 tests (the Defects4J relevant tests are one class, `LocaleUtilsTest`; the failing test is `testParseAllLocales`), 208 statements in the spectrum. The Jira report is 8,265 characters and contains **49 stack-frame lines from an unrelated project** (Maven Surefire, TestNG, `com.scispike...`), which is pure noise for retrieval.

Standard pipeline, k = 50% (paper-style IR).

Method	Tests	Statements	IR rank of buggy file	Tie group at the buggy line	Rank w / a / b	TOP@1/3/5 (w)	MRR (w)
Ochiai	14	208	**23**	3 lines (1 buggy)	4 / 3.0 / 2	0 / 0 / 1	0.250
ISBL + Ochiai	14	208	23	3 lines (1 buggy)	4 / 3.0 / 2	0 / 0 / 1	0.250

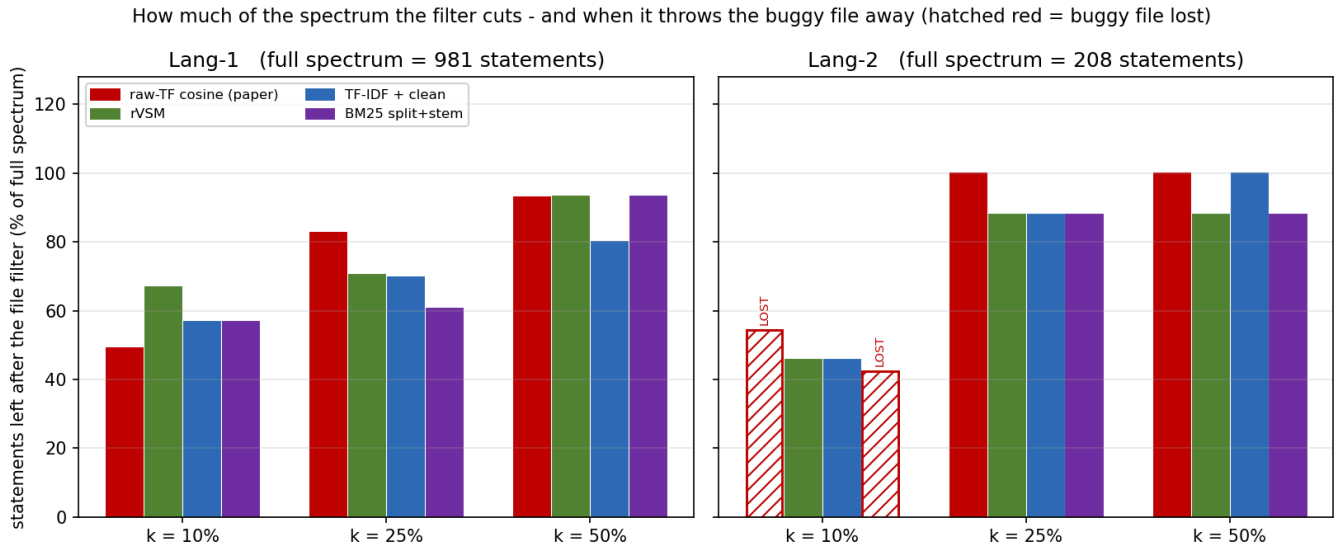
IR facts: cosine of the buggy file 0.1112, but the top file (`JavaVersion.java`) scores 0.3169; the filter removes **0%** of the statements at k = 50%, because the 14 tests touch few files.

File level: see the combined table in section 7 (right column). Highlights for Lang-2: the paper's cosine ranks `LocaleUtils.java` **23rd**; removing the pasted trace moves it to 10th; every TF-IDF/rVSM variant with the clean query

ranks it **1st**. Identifier splitting plus stemming is **not** automatically better: TF-IDF drops from 1st to 5th, rVSM from 1st to 2nd, BM25 from 8th to 26th when switched on.

False exclusion on Lang-2 (statements left, and whether the buggy line survived).

IR model	k = 50%	k = 25%	k = 10%
raw-TF cosine (paper)	208, kept	208, kept	113, **LOST** (only 1 test left; bug cannot be found)
rVSM / paper	183, kept	183, kept	95, kept
TF-IDF sublinear / split+stem + clean	208, kept	183, kept	95, kept
BM25 / split+stem	183, kept	183, kept	88, **LOST**
Dirichlet LM / split+stem	120, kept	25, **LOST**	25, **LOST**



Statement level: Lang-2.

Method	Stmts	Tests	Best buggy rank w / a / b	TOP@1/3/5 (w)	MRR w	MRR a	MAP a	nDCG@10 a	EXAM a	Wasted a
SFL: Ochiai (= the 9 equivalent formulas)	208	14	4 / 3.0 / 2	0/0/1	0.250	0.333	0.333	0.500	0.0144	2.0
SFL: Wong2	208	14	204 / 201 / 198	0/0/0	0.005	0.005	0.005	0.000	0.9663	200
ISBL, raw-TF (paper), k = 50% / 25%	208	14	4 / 3.0 / 2	0/0/1	0.250	0.333	0.333	0.500	0.0144	2.0
ISBL, raw-TF (paper), **k = 10%**	113	1	**buggy line lost**	0/0/0	**0**	**0**	**0**	**0**	-	-
ISBL, TF-IDF + clean, k = 50%	208	14	4 / 3.0 / 2	0/0/1	0.250	0.333	0.333	0.500	0.0144	2.0
ISBL, TF-IDF + clean, **k = 10%**	**95**	14	4 / 3.0 / 2	0/0/1	0.250	0.333	0.333	0.500	0.0144	2.0
Hybrid, statement text (`tie_text`)	208	14	4 / 3.0 / 2	0/0/1	0.250	0.333	0.333	0.500	0.0144	2.0
Hybrid, linear blend (a = 0.3, b = 0.3)	208	14	7 / 6.0 / 5	0/0/0	0.143	0.167	0.167	0.356	0.0288	5.0

Why the hybrid did not help on Lang-2. The buggy line 92 ties with lines 89 and 93, none of which shares a word with the report (text score 0). Line 94, `throw new IllegalArgumentException("Invalid locale format")`, is ranked first because only two tests run it; it comes from the validation code that exists in both versions and is *not* the bug. The report does not talk about the missing check in terms of these lines, so there is no signal to break the tie. The linear blend, in contrast, lets file and text scores overrule the tests and **loses** (rank 7).

Reading Lang-2.

1. A noisy report (49 foreign stack frames) breaks the paper's raw-count cosine (rank 23); cleaning plus idf weighting repairs it (rank 1).
2. The benefit of the better IR is **safety at small k**: at k = 10% the paper's IR loses the bug, TF-IDF/rVSM keep it while cutting 54% of the statements.
3. Tie-breaking by words is not a universal fix; it only helps when the buggy lines share words with the report (true on Lang-1, not on Lang-2).

---

9. Cosine similarity audit

Question from the review: *is our cosine similarity right?* Script: `scripts/run/cosine_audit.py`; data: `results/experiments/cosine_audit_*.csv`.

Check	Lang-1	Lang-2
Largest difference to scikit-learn `cosine_similarity`	4.2e-17	5.6e-17
Largest difference to SciPy cosine distance	8.3e-17	1.1e-16
Same ranking as scikit-learn	yes	yes
Values in [0, 1]; self-similarity = 1	yes; 1.0	yes; 1.0
Symmetric; unchanged if the query counts are multiplied by 7; zero vector -> 0	yes	yes

Verdict: the arithmetic is right. The weighting is the problem. Where the top-ranked (wrong) file gets its score:

Bug	Top file (cosine)	Words that produce its score	Share of the score from words found in > 50% of all files	Buggy file: rank, cosine, top words
Lang-1	`Validate.java` (0.156)	not 69%, string 16%, method 5%, check 4%	**91%**	`NumberUtils.java`: 8, 0.120; not 22%, string 18%, numberutils 16%, integer 11%, hex 8%
Lang-2	`JavaVersion.java` (0.317)	java 84%, org 7%, apache 6%	**99.9%**	`LocaleUtils.java`: 23, 0.111; locale 43%, java 35%, org 6%, apache 5%

In plain words: raw counts reward ordinary words ("not", "string", and in Lang-2 "java/org/apache" from the pasted trace). TF-IDF gives those words almost no weight. (The paper's stop-word list is not published; ours does not contain "not", which may differ from the authors'.)

10. Speed: where the time goes

Stage (Lang-1, 173 tests)	Before	After
Coverage collection, JVM total	9.5-9.6 s (JaCoCo re-analysing all classes after every test: 96-98% of it)	**1.15 s** (analyses only executed classes; 73% is still analysis)
Throughput	18 tests / s	**150 tests / s**
All other stages together (IR, spectrum, scoring, metrics)	about 0.2 s	about 0.2 s
Lang-2 (14 tests)		0.54 s JVM total

The coverage output is **identical** to the slow version (compared line by line on all 403 lines). Running the full Lang test suite (2,280 tests) took 232 s before the change.

11. Other findings and quality checks

- **Formulas:** 11 SFL formulas, 9 give identical rankings on Lang-1 and Lang-2; Wong2 is far worse (it ignores normalisation).
- **Tests:** 104 automated unit tests (cosine, TF-IDF checked against scikit-learn, BM25, rVSM, hybrid keys, metrics, answer key, query cleaning, timing) pass on Python 3.7 and 3.12.
- **Preparing a new bug** (Lang-2): find the fixing commit in the Apache history that accepts the Defects4J patch -> check it out -> apply the patch (gives the buggy source) -> compile on JDK 8 -> fetch the Jira report -> run the tests -> **require that the Defects4J trigger test fails**. Two Windows issues were found and fixed: the Defects4J clone had CRLF patch files (normalised to LF), and `TypeUtilsTest` cannot be compiled by javac 8 (excluded, as for Lang-1).
- **Answer key** is read directly from the Defects4J patch; for an omission bug it is the first line after the gap.

12. Limits and honest caveats

- **Few bugs.** Three bugs of one project (two detailed here); the paper uses 373 over six projects. Nothing here confirms or contradicts the paper's tables.

- **Post-hoc choice.** `tie_text` looked best among 12 hybrid variants evaluated on the same three bugs; confirm it on more bugs before relying on it. It narrows the uncertainty of a tie; it does not guarantee rank 1 (on Lang-1 the best-case rank 1 became 2).
- **Our own extension.** The hybrid tie-break is not part of the ISBL paper. The BLUiR-like model uses regular expressions, not a Java parser. BugLocator's similar-bug history, stack-trace boosting, call-frequency tie-breaking and learned retrieval were **not** implemented.
- **Assumptions the paper leaves open:** how ties are counted, DStar's exponent (we use 2), its stop-word list.
- **Coverage tool:** JaCoCo line coverage instead of Defects4J's own coverage tooling; only the Defects4J relevant tests are run.
- **Literature scan:** abstracts and search snippets, not the full papers (`docs/literature.md`).

13. How to reproduce and where things are

```
cd <path-to-this-repository>
. .\scripts\setup\env.ps1                              # Java 8 + Python 3.7 on the PATH
python -m isbl.baseline.pipeline configs\lang1_k50.json --explain          # standard pipeline, Lang-1 (printed
tables)
python -m isbl.baseline.pipeline configs\bugs\Lang-2.json --explain        # same on Lang-2
python scripts\bugs\prepare_lang_bug.py 2 3            # one-time: prepare Lang-2, Lang-3 (Maven + internet)
python scripts\run\cosine_audit.py                     # section 9
python scripts\run\run_experiments.py                  # sections 7-8 tables, about 50 s
envs\modern312\Scripts\python scripts\run\build_excel_experiments.py   # results\Lang-1-2-3_experiments.xlsx
python -m pytest -q tests                              # 104 tests
C:\Users\Prasun\anaconda3\python.exe scripts\run\make_report_figures.py   # charts of this report
dot -Tpng -Gdpi=130 docs\images\pipeline_new.dot -o docs\images\pipeline_new.png   # diagrams (Graphviz)
```

What	Where
IR models, preprocessing	`isbl/baseline/ir.py`, `ir_models.py`
Spectrum, coverage runner	`isbl/baseline/spectrum.py`, `java/CoverageRunner.java`
Formulas, ranking, hybrid	`sfl.py`, `ranking.py`, `hybrid.py`
Metrics	`isbl/evaluation/metrics.py`, `diagnostics.py`
Experiments	`isbl/experiments/engine.py`, `scripts/run/run_experiments.py`
Results	`results/raw/` (pipeline), `results/experiments/` (comparisons), `results/*.xlsx`
More documents	`docs/experiments.md` (all 3 bugs), `docs/literature.md`, `docs/reproduction.md`, `README.md`

14. References (full list with links: `docs/literature.md`)

- Li, Yue, Lan, Cui. *Statement-level Software Bug Localization based on Information Retrieval and Spectrum*. IEEE SMC 2023. (the paper we reproduce)
- Zhou, Zhang, Lo. *Where should the bugs be fixed?* (BugLocator, rVSM). ICSE 2012.
- Saha et al. *BLUiR* (structured information retrieval). ASE 2013.
- Le, Oentaryo, Lo. *Information Retrieval and Spectrum Based Bug Localization: Better Together*. FSE 2015.
- Sarhan, Vancsics, Beszedes. *Method calls frequency-based tie-breaking strategy for software fault localization* (arXiv 2108.11031): about 54-56% of buggy methods in Defects4J tie with others.
- Surveys: *When Deep Learning Meets IR-based Bug Localization* (arXiv 2505.00144); *Spectrum-based Software Fault Localization: A Survey* (arXiv 1607.04347); Zou et al., *An Empirical Study of Fault Localization Families and Their Combinations* (arXiv 1803.09939).

Experiments: better IR models, more metrics, other Lang bugs, hybrid ranking

Written for: the project owner and the faculty reviewer. Date of results: 2026-10-25.
Scope: Lang-1, Lang-2 and Lang-3 only (3 bugs of Defects4J's 395; the ISBL paper uses 373). The differences below
are small counts and indicative only; no significance is claimed. No parameter was tuned on these bugs: every configuration was fixed in advance (see `isbl/experiments/engine.py`).

1. What was asked and what was done

Request	Done
Add more metrics for comparison	File level: Top@1/5/10, MRR, MAP, nDCG@10, recall@k% of files. Statement level: TOP@1/3/5 (worst-case and average ties), MRR (3 tie rules), MAP, nDCG@10, recall@10, EXAM, wasted effort, win/draw/lose vs Ochiai, bootstrap CI, Wilcoxon p (`isbl/evaluation/metrics.py`)
Check the cosine similarity properly	`scripts/run/cosine_audit.py`: arithmetic verified against scikit-learn and SciPy; properties verified; per-term diagnosis of why the wrong file wins (section 3)
Try tf-idf and more proper IR measures	12 models in `isbl/baseline/ir_models.py`: raw-tf cosine (paper), authors' tf-idf, smoothed tf-idf, sublinear tf-idf, BM25, BugLocator rVSM, Dirichlet language model, Jaccard, containment, LSI, BLUiR-style structured BM25, rank fusion; plus options: identifier splitting, stemming, licence-header removal, stack-trace cleaning, class-name bonus
Webscrape relevant papers for new metrics, add to the code	`docs/literature.md` (11 sources, abstracts/snippets only) and the code additions it maps to
Find more types of bugs	Three different bug types (below), plus 9 more SFL formulas and a hybrid ranking that uses the bug report to break spectrum ties
Run on other Lang bugs	Lang-2 and Lang-3 prepared with `scripts/bugs/prepare_lang_bug.py` (commit mapping, Defects4J patch, JDK 8 compile, Jira report, coverage, trigger-test sanity check)

2. The three bugs

Bug	Jira	Type	Buggy lines (answer key)	Tests	Statements	Report
Lang-1	LANG-747	wrong condition (counts hex digits instead of checking the value)	`NumberUtils.java` 467, 468, 471	173	981	662 chars
Lang-2	LANG-879	missing validation (omission of a guard clause)	`LocaleUtils.java` 92 (line after the missing guard)	14	208	8,265 chars, contains 49 stack-frame lines of an **unrelated** project
Lang-3	LANG-693	missing precision branches (omission of conditional guards)	`NumberUtils.java` 593, 597, 601, 605	172	979	480 chars

How a bug is built (all on Windows, no Defects4J command line): find the fixing commit in the Apache history whose tree
accepts Defects4J's `N.src.patch`, check it out in a `git worktree`, apply the patch (it turns the fixed source into the buggy source),
keep the fixed tests, compile with Maven on JDK 8, run the Defects4J relevant tests under the per-test JaCoCo runner, and **require that every Defects4J trigger test fails**
(otherwise the bug is rejected). Lessons: the Windows clone of Defects4J has CRLF patch files, which had to be normalised to LF; test files javac 8 cannot compile
(`TypeUtilsTest`) are removed, as for Lang-1. The answer key is read directly from the Defects4J patch (`isbl/baseline/answer_key.py`);
for an omission bug it is the first buggy line after the gap (same rule as before).

3. Cosine audit (the paper's formula 1, raw term frequency)

Check	Result
Our cosine vs scikit-learn `cosine_similarity`	max abs difference 4e-17 to 6e-17 on all three bugs; identical ranking
Our cosine vs SciPy `cosine` distance	max abs difference 8e-17 to 1.1e-16
Properties	values in [0, 1]; self-similarity 1; symmetric; unchanged when the query counts are multiplied by 7; zero vector gives 0

So the arithmetic is right; the **weighting** is the problem. For the top-ranked (wrong) file, the share of its cosine that comes from words occurring in more than half of all files is 91% (Lang-1: `Validate.java`, driven by "not", "string"), 100% (Lang-2: `JavaVersion.java`, driven by "java", "org", "apache" from the pasted trace) and 88% (Lang-3: `MethodUtils.java`, driven by "method"). The buggy file is ranked 8th, 23rd and 16th. The paper's stop-word list is not published; ours does not contain "not" (a likely difference). Details: `results/experiments/cosine_audit_*.csv`.

4. File level: which IR configuration ranks the buggy file best?

(`|` in names is shown as `/`.) Rank of the buggy file among 108 files, Lang-1 / Lang-2 / Lang-3. Preprocessing: `paper` = Java keywords and stop words removed; `split+stem` = also licence header removed, identifiers split, Snowball stemming; `+clean` = stack-frame lines removed from the report; `+name` = bonus for a class name that occurs in the report.

IR configuration	buggy-file rank	Top@1	Top@5	MRR	MAP	nDCG@10	recall@10% files
bm25 / split+stem +name	1/1/1	3	3	1.000	1.000	1.000	1.00
rvsm / paper	1/1/1	3	3	1.000	1.000	1.000	1.00
rvsm / split+stem +clean	1/1/1	3	3	1.000	1.000	1.000	1.00
tfidf_authors / none	1/1/1	3	3	1.000	1.000	1.000	1.00
tfidf_smooth / paper	1/1/1	3	3	1.000	1.000	1.000	1.00
tfidf_sublinear / paper	1/1/1	3	3	1.000	1.000	1.000	1.00
tfidf_sublinear / split+stem +clean	1/1/1	3	3	1.000	1.000	1.000	1.00
tfidf_sublinear / split+stem +name	1/1/1	3	3	1.000	1.000	1.000	1.00
rvsm / split+stem	1/2/1	2	3	0.833	0.833	0.877	1.00
bluir_like / split+stem +clean	1/3/1	2	3	0.778	0.778	0.833	1.00
lm_dirichlet / split+stem +clean	1/5/1	2	3	0.733	0.733	0.796	1.00
lsi / split+stem	1/5/1	2	3	0.733	0.733	0.796	1.00
tfidf_sublinear / split+stem	1/5/1	2	3	0.733	0.733	0.796	1.00
rrf(bm25,tfidf,rvsm) / split+stem	1/6/1	2	2	0.722	0.722	0.785	1.00
bm25 / paper	1/8/1	2	2	0.708	0.708	0.772	1.00
bm25 / split+stem +clean	1/9/1	2	2	0.704	0.704	0.767	1.00
bluir_like / split+stem	1/11/1	2	2	0.697	0.697	0.667	1.00
bm25 / split+stem	1/26/1	2	2	0.679	0.679	0.667	0.67
lm_dirichlet / split+stem	1/35/1	2	2	0.676	0.676	0.667	0.67
jaccard / paper	1/5/2	1	3	0.567	0.567	0.673	1.00
containment / paper	1/8/2	1	2	0.542	0.542	0.649	1.00
tf_cosine / paper +clean	8/10/16	0	0	0.096	0.096	0.202	0.67
tf_cosine / paper (the ISBL paper)	**8/23/16**	0	0	0.077	0.077	0.105	0.33

Reading (3 bugs, so a ceiling effect hides differences between the top rows):
- Every idf-weighted model (tf-idf variants, rVSM, LSI) ranks the buggy file 1st on Lang-1 and Lang-3. The paper's raw-tf cosine is last.
- On Lang-2 the pasted trace hurts: removing stack frames moves the buggy file 23 -> 10 (raw tf), 5 -> 1 (tf-idf), 2 -> 1 (rVSM), 11 -> 3 (BLUiR-like), 35 -> 5 (LM); BM25 improves only 26 -> 9.
- Identifier splitting and stemming are not an automatic win: on Lang-2 the buggy file falls from 1st to 5th (tf-idf), from 1st to 2nd (rVSM) and from 8th to 26th (BM25) when `split+stem` is switched on.
- The worst cases are rank 35 (Dirichlet LM, `split+stem`, Lang-2) and rank 26 (BM25, `split+stem`, Lang-2); recall@25% of files is 0.67 for the LM, 1.00 for the others.

4b. False exclusion: does the top-k filter throw the buggy file away?

This is the failure mode the ISBL paper itself names (Math-51 there). "lost" = bugs (of 3) whose buggy file is not among the kept files, so the bug cannot be localised;
"cut" = mean share of the spectrum's statements removed (a lost bug inflates the cut, so read the two together).

IR configuration	k = 10%: lost / cut	k = 25%: lost / cut	k = 50%: lost / cut
tf_cosine / paper (the ISBL paper)	**2/3 / 62%**	0/3 / 19%	0/3 / 5%
tf_cosine / paper +clean	1/3 / 47%	0/3 / 19%	0/3 / 5%
tfidf_sublinear / paper	0/3 / 47%	0/3 / 24%	0/3 / 14%
tfidf_sublinear / split+stem +clean	0/3 / 46%	0/3 / 27%	0/3 / 15%
rvsm / paper	0/3 / 44%	0/3 / 27%	0/3 / 15%
bluir_like / split+stem +clean	0/3 / 46%	0/3 / 21%	0/3 / 9%
bm25 / paper	0/3 / 46%	0/3 / 20%	0/3 / 7%
bm25 / split+stem	1/3 / 48%	0/3 / 29%	0/3 / 11%
lm_dirichlet / split+stem	1/3 / 71%	1/3 / 69%	0/3 / 51%

(all 23 configurations: `results/experiments/false_exclusion.csv`, sheet `False_Exclusion`.) With the paper's raw-tf IR the filter has to stay at k >= 25% to be safe on these bugs and then removes only 19% (k = 25%) or 5% (k = 50%) of the statements; a tf-idf, rVSM or BLUiR-style ranking keeps the buggy file even at k = 10% while cutting 44-47% of the spectrum. **That, not a different statement ranking, is where a better IR step pays off here.**

5. Statement level

Best buggy-statement rank per bug (Lang-1/2/3) with tied statements averaged; "worst" counts a tied statement at the end of its tie group.

Method	rank per bug (ties averaged)	TOP@1/3/5 (worst-case ties)	MRR worst	MRR avg	MAP avg	EXAM avg	W/D/L vs Ochiai
SFL: Ochiai (also DStar, Tarantula, Jaccard, Op2, Barinel, Kulczynski2, Ochiai2, Sorensen-Dice, DStar3)	3.5 / 3 / 3.5	0/0/2	0.206	0.302	0.327	0.0072	0/3/0
SFL: Wong2	628 / 201 / 631	0/0/0	0.003	0.003	0.004	0.7503	0/0/3
ISBL, raw-tf IR (the paper), k = 0.5, Ochiai	3.5 / 3 / 3.5	0/0/2	0.206	0.302	0.327	0.0072	0/3/0
ISBL, tf-idf + clean, k = 0.5, Ochiai	3.5 / 3 / 3.5	0/0/2	0.206	0.302	0.327	0.0072	0/3/0
Hybrid tie_file (+ file IR score), k = 0.5	3.5 / 3 / 3.5	0/0/2	0.206	0.302	0.329	0.0072	0/3/0
Hybrid tie_text (+ statement text), k = 0.5	**2 / 3 / 2**	**0/2/3**	0.417	0.444	0.376	0.0062	2/1/0
Hybrid tie_file_text, k = 0.5	**2 / 3 / 2**	**0/2/3**	0.417	0.444	0.377	0.0062	2/1/0
Hybrid linear a = 0.3, k = 0.5	3.5 / 3 / 3.5	0/0/2	0.206	0.302	0.329	0.0072	0/3/0
Hybrid linear a = 0.3, b = 0.3, k = 0.5	2 / 6 / 5	0/1/2	0.281	0.289	0.297	0.0120	1/0/2

(The hybrid rows use TF-IDF with split+stem and +clean as the IR model; all eight IR models in `HYBRID_IR` behave the same, see `results/experiments/statement_level_summary.csv`.)

Findings:
1. **No SFL formula beats Ochiai here** (Wong2 is far worse). Lines with identical (ef, ep, nf, np) get identical scores under every formula, so formula choice cannot break the ties that dominate these bugs (tie groups of 6, 3 and 4).
2. **A better file ranking does not change the statement ranking when the buggy file survives the filter** (it does for every model at k = 50%): the ties are inside the buggy file. What changes is how aggressively the spectrum can be cut safely (section 4b): at k = 50% the paper's raw-tf IR removes 6.9% / 0% / 7.2% of the statements of Lang-1/2/3 and tf-idf + clean removes 19.9% / 0% / 25.4%; at k = 10% tf-idf + clean removes 43.1% / 54.3% / 40.3% with the buggy file kept, while the paper's raw-tf IR loses the buggy file on 2 of 3 bugs (their ISBL rows then have no buggy statement to rank).
3. **Exact-tie breaking with the report's words on each statement helps**: expected rank 3.5 -> 2 on Lang-1 and Lang-3 (worst-case 6 -> 2 and 5 -> 2), no change on Lang-2. TOP@3 goes from 0 to 2 bugs under worst-case ties. Caveat: the *best*-case rank on Lang-1 gets worse (1 -> 2): the method narrows the uncertainty of the tie, it does not guarantee rank 1.
4. **How scores are combined matters**: a lexicographic key (SFL score first, text only for exact ties) helps; a normalised

linear blend with the same information is worse than plain Ochiai (1 win, 2 losses).
5. The file score as a tie-breaker changes nothing, because tied statements share a file.

Selection caveat: `tie_text` / `tie_file_text` were the best of 12 hybrid variants evaluated on these same three bugs (all variants are in the CSV and in the `Hybrid_Modes` sheet). Treat the result as a hypothesis to confirm on more bugs.

6. Reproduce

```
. .\scripts\setup\env.ps1
python scripts\bugs\prepare_lang_bug.py 2 3     # one-time: needs Maven, internet (Maven Central, Apache Jira),
JDK 8 in tools\
python scripts\run\cosine_audit.py              # cosine correctness and diagnosis
python scripts\run\run_experiments.py           # all tables, about 50 s (coverage is cached)
envs\modern312\Scripts\python scripts\run\build_excel_experiments.py   # results\Lang-1-2-3_experiments.xlsx
python -m pytest -q tests                       # 104 tests
```

7. Limits and what is not done

- Three bugs from one project; the other 62 Lang bugs and the other five projects are not run (the preparation script supports `all`, but each bug needs a Maven compile and downloads).
- IR/hybrid configurations are not tuned (by design), and with n = 3 the bootstrap CI and Wilcoxon columns are not informative.
- Not implemented: BugLocator's similar-bug history, stack-trace boosting, call-frequency tie-breaking, learned/semantic retrieval. The BLUiR-style model uses regex field extraction, not a Java parser.
- Relevant tests only (Defects4J's list); the full-suite spectrum was only compared for Lang-1.
- Statement text score and hybrid modes are our own exploratory extension, not part of the ISBL paper.

Literature scan: metrics and methods for IR-based + spectrum-based bug localisation

Collected on 2026-10-25 by web search (WebSearch, abstracts and search snippets only; **the full papers were not read**).
Each row says what the source reports and what, if anything, was added to this code base. Formula definitions that the search results did not show (Barinel, Op2, Kulczynski2, Ochiai2, Sørensen-Dice) are the standard ones from the SBFL literature and are covered by unit tests, but are not verified against the cited surveys here.

1. Evaluation metrics

Source	What it reports	Added here
When Deep Learning Meets IR-based Bug Localization: A Survey, arXiv 2505.00144	Of 61 primary studies, 59 use **MAP**; MAP, **MRR** and **Top-k** are the usual metrics; AUC/precision/recall/F-measure appear only occasionally	`average_precision`, `ap_over_all_relevant` (MAP), `ndcg_at_k`, `recall_at_k`, `precision_at_k` in `isbl/evaluation/metrics.py`; MAP, MRR, Top@1/5/10, nDCG@10, recall@k% at **file level** in the experiment tables
FlexFL, arXiv 2411.10714 and related snippets	Top-N with N = 1, 3, 5 is close to what developers expect; **EXAM** = share of statements examined before the first buggy one	`exam_score` (one fixed denominator for every method, the full-spectrum statement count), `wasted effort = rank - 1`, TOP@1/3/5 kept as in the ISBL paper
Literature review of bug-localisation approaches, arXiv 2212.11774	Survey of empirically evaluated approaches	background only
ISBL paper (Li et al., SMC 2023), section IV-A	**WIN / DRAW / LOSE** per bug against Ochiai/DStar	`win_draw_lose` (by best buggy-statement rank), plus `bootstrap_ci` and `wilcoxon_p`. With only 3 bugs these are reported for completeness, not as evidence

2. Information-retrieval models for ranking files

Source	What it reports	Added here (`isbl/baseline/ir_models.py`)
Zhou, Zhang, Lo, *Where should the bugs be fixed?* (BugLocator), ICSE 2012, DOI 10.1109/ICSE.2012.6227210	**rVSM**: a tf-idf variant with a size-based correction that favours larger files; BugLocator also uses similar previously fixed bugs	`rvsm` (log-tf x log-idf cosine times a logistic file-length bonus). The similar-bug history part is **not** implemented (no report history available)
Saha et al., *BLUiR*, ASE 2013	Structured IR: class, method and variable names and comments of each file are separate fields; two fields of the bug report	`bluir`: BM25 between {summary, description} and {class name, method names, identifiers, comments}, summed. Field extraction is a regex heuristic, not a Java parser
A feature-location study (snippet only; PDF, authors/venue not visible in the snippet)	On four open-source projects BM25 and BM25F were consistently better than VSM, a unigram model and LDA	`bm25` (Okapi, k1 = 1.2, b = 0.75), `lm_dirichlet` (unigram query likelihood, mu = 2000), `lsi` (truncated SVD)
Purdue, SCAM 2012: effect of stemming on keyword-based source-code retrieval	The benefit of stemming depends on other factors such as whether tf-idf already filters common terms	`stem` option (Snowball English stemmer). Evaluated both with and without it, not assumed to help
Search snippets on tf-idf / cosine bug localisation	Plain VSM with tf-idf and cosine plus Porter stemming is the common baseline	`tfidf_authors` (the repo's `vsm.py`), `tfidf_smooth`, `tfidf_sublinear` (scikit-learn semantics, unit-tested against `TfidfVectorizer`)
General IR practice	Jaccard / term containment as simple set baselines; reciprocal-rank fusion for combining rankings	`jaccard`, `containment`, `rrf`

3. Combining bug reports with program spectra

Source	What it reports	Added here
Le, Oentaryo, Lo, *Information Retrieval and Spectrum Based Bug Localization: Better Together*, FSE 2015, DOI 10.1145/2786805.2786880	A multi-modal method using both bug reports and spectra. On 157 real bugs from four systems it beat the baselines by at least 47.62% / 31.48% / 27.78% / 28.80% on Top 1 / Top 5 / Top 10 / MAP (abstract)	Motivates `hybrid.py`. Our combination is much simpler (lexicographic tie-breaking and blends); it is **not** a reimplementation of their method
Dao, Zhang, Meng, *How does execution information help with IR-based bug localization?*, 2017	Studies how execution information helps IR-based localisation (same 157-bug data set)	background
ISBL paper (Li et al., SMC 2023)	IR filters files, then the spectrum is reduced	the baseline in `pipeline.py`

4. Ties in spectrum-based fault localisation

Source	What it reports	Added here
Sarhan, Vancsics, Beszédes, *Method calls frequency-based tie-breaking*, arXiv 2108.11031	About 54-56% of buggy methods in Defects4J share their suspiciousness with at least one other method (average tie group roughly six); their call-frequency tie-breaker removed ties completely in 72-73% of cases	Matches what we see on Lang-1 (a tie group of six). We break ties with bug-report text instead (`tie_text`); their call-frequency idea is **not** implemented
Sarhan, Beszédes, a ranking formula that breaks ties only	Raised the number of cases where the faulty element is ranked first by 13-23% on six Defects4J programs	background for the lexicographic design (a secondary key may only reorder exact ties)

5. SFL formula families

Source	What it reports	Added here (`isbl/baseline/sfl.py`)
Spectrum-based Software Fault Localization: A Survey, arXiv 1607.04347	Survey of SBFL techniques	Tarantula, Jaccard, Op2, Barinel, Kulczynski2, Wong2, Ochiai2, Sørensen-Dice, DStar(3) next to Ochiai and DStar(2)
Zou et al., *An Empirical Study of Fault Localization Families and Their Combinations*, arXiv 1803.09939	Compares fault-localisation families and their combinations	background for combining families; not implemented

6. Ideas found but not implemented

- **Stack-trace boosting.** BRTracer+ boosts files named in the top stack-trace frames (search snippet); another study found reports containing stack traces can *hurt* some IR tools. Boosting was not implemented. What we did observe: Lang-2's report contains 49 stack-frame lines from an *unrelated* project (Maven Surefire / TestNG), Lang-1 and Lang-3 contain none; stripping those lines (`clean_query`, `strip_stack_traces`) improves the IR ranking on Lang-2 (e.g. TF-IDF buggy-file rank 5 -> 1).
- **Similar-bug history** (BugLocator), **code-change history** (BLIA), **query reformulation**, **deep-learning retrieval**, **call-frequency tie-breaking**: all need data or models beyond this single-project set-up.

7. Cosine similarity audit (done in this repo)

`scripts/run/cosine_audit.py` checks the paper's cosine (formula 1) on Lang-1/2/3:
our implementation equals scikit-learn's `cosine_similarity` and SciPy's cosine distance to 1e-16, stays in [0, 1], is symmetric and
scale-invariant, and returns 0 for a zero vector. The weakness is the **weighting**, not the arithmetic: for the top-ranked (wrong) file, 88-100% of the cosine comes from terms present in more than half of all files ("not", "string", "java", "method").
See `results/experiments/cosine_audit_*.csv`.

Reproduction status - Lang-1 ONLY (scope decided 2026-10-14: complete working of this single bug)

Not a reproduction of the paper's 373-bug numbers; nothing here is claimed to match them.

1. Environment: Windows 11 native; JDK 8 (Temurin 8u504); Python 3.7.16 (legacy) / 3.12.4 (modern); JaCoCo 0.8.12; Defects4J v1.4.0 metadata.
2. Commits: see docs/baseline.md. 3. Defects4J version: 1.4.0 (metadata only; the Defects4J CLI was not run, see environment-audit.md).
3. Bug evaluated: Lang-1 (LANG-747, NumberUtils.createNumber hex handling; buggy lines 467, 468, 471 of NumberUtils.java).
4. Bug report: Apache Jira LANG-747, summary + description -> data/bug_reports/Lang-1.json.
5. Source processing: all 108 .java files under src/main/java of the buggy checkout.
6. IR (now follows paper II-A): lowercase 2+ char word tokens, Java keywords and stop words removed, raw word-frequency vectors, cosine (formula 1). The authors' repo script (TF-IDF, no removal) is kept as ir_mode "tfidf". Buggy file rank: 8/108 (paper-style) vs 1/108 (TF-IDF). Paper does not say how identifiers are split; we do not split them (assumption).
7. k: fraction of files, ceil(k*N); k=0.5 headline, 0.1..0.9 swept (results/raw/Lang-1_tf_*_sweep_k.csv). The buggy file is inside the top-k for every k from 0.1 to 0.9 (rank 8 of 108).
8. Spectrum: JaCoCo line coverage per test method (executable source line = statement; main classes only). Two test scopes: Defects4J relevant tests (173 tests) and the full Lang suite (2290 tests). Failing = Defects4J trigger test only (TestLang747); 7 other failures in the full suite (locale/JDK 8 vs 7 differences) are removed, as Defects4J does. Abstract test base classes are skipped.
9. Reduction (paper II-B): drop coverage outside the top-k files, then drop tests that cover nothing left; nf is counted on the reduced test set.
10. Ochiai: ef/sqrt((ef+nf)(ef+ep)), 0 if denominator 0. 12. DStar: ef^2/(ep+nf), inf if ep+nf=0 and ef>0 (exponent 2 is an assumption).
11. Ranking: descending score; ties -> worst-case rank by default; worst/average/best are all recorded.
12. Metrics: rank = best-ranked buggy statement; TOP@N counts bugs with rank <= N; MRR = mean(1/rank). Answer key = lines changed by the fix diff (git diff buggy fixed); evaluation only.
13. Results (k=0.5, paper-style IR). Rank worst/average/best:
    relevant tests (173): Ochiai 6/3.5/1 | ISBL-Ochiai 6/3.5/1 | DStar 6/3.5/1 | ISBL-DStar 6/3.5/1 (statements 981 -> 913)
    full suite (2273): Ochiai 6/3.5/1 | ISBL-Ochiai 6/3.5/1 | DStar 6/3.5/1 | ISBL-DStar 6/3.5/1 (statements 11102 -> 6593)
    Worst-case ties: TOP@1/3/5 = 0/0/0, MRR 0.167. Best-case ties: rank 1.
    Why identical: six lines of NumberUtils.java (462, 463, 467, 468, 471, 474) are executed by exactly the same four tests - the failing TestLang747 and three passing tests (testCreateNumber, testCreateNumberMagnitude, testIsNumber) - so they have identical (ef,ep,nf,np) = (1,3,0,166) and tie at Ochiai 0.50 / DStar 0.33. Three of the six (467, 468, 471) are the buggy lines. No test can separate them because no test executes one of these lines without the others. The IR filter removes unrelated statements (up to 41%) but cannot split tied lines inside the buggy file, so the best statement rank does not change. On this bug ISBL = plain Ochiai/DStar. (An earlier version of this note wrongly said ep = 0; the --explain output corrects that.)
    16-18. Paper comparison: not possible with one bug. Open discrepancies: tie handling and DStar exponent unspecified in the paper; BugLocator/Gharibi filters not implemented; JaCoCo instead of Defects4J's coverage tool; commit hashes remapped to GitHub history; Maven instead of Defects4J's Ant; TypeUtilsTest excluded (javac 8).
    Note: an earlier draft run used the full suite WITHOUT the trigger-test filter and showed ISBL beating DStar (rank 67 -> 33). That was an artifact of 17 spurious failing tests (abstract classes run directly + JDK/locale failures) and is superseded by the numbers above.
    Tests: 49 unit tests pass on Python 3.7 and 3.12.

Run: .\scripts\run\demo.ps1 | python -m isbl.baseline.pipeline configs\lang1_k50.json (relevant tests) or configs\lang1_k50_alltests.json (full suite, ~2.5 min on first run).

Profiling and extra metrics (added 2026-10-14)

Every run now logs per-stage wall-clock time (results/raw/_timing.csv, printed table) and, for each method row, IR / spectrum / localisation diagnostics (results/raw/.csv and .json).
Where the time goes (Lang-1, k=0.5):
scope	total	coverage collection (JVM+JaCoCo)	everything else
relevant tests (173)	9.9 s	9.7 s (98%)	0.2 s
full suite (2280)	231.9 s	231.1 s (99.6%)	~0.8 s
Inside the JVM run, the JaCoCo class analysis that is repeated after EVERY test takes 97.7% (relevant) / 93.5% (full suite) of the time; running the project's tests takes only 0.6% / 6.0% (13.9 s for 2280 tests). Candidate optimisation (not yet done): analyse only classes whose probes were hit by the test, or compute the probe->line map once per class, instead of re-analysing all classes per test. IR (0.12 s), spectrum construction, SFL scoring and metrics are all negligible at one-bug scale; IR and SFL scoring would only matter at hundreds of bugs.
New metrics (per row): IR - buggy-file rank/percentile (= smallest k that keeps it), buggy-file cosine, top-1 cosine, top1-top2 gap, score entropy (candidate adaptive-k signal), files with cosine>0, recall@k; spectrum - matrix density, avg statements/test, avg tests/statement; reduction - % statements and tests removed; localisation - exam score, wasted effort, mean/median buggy rank, #distinct scores, tie-group size containing the best buggy line and how many of it are buggy, buggy-line score vs top score, TOP@10.
Lang-1 reading: only 5 distinct score values over 913 statements; the best buggy line sits in a 6-way tie (3 of 6 buggy), so exam score is tiny (0.0066) but TOP@N is 0 under worst-case ties - the problem is ties, not the retrieval.
Tests: 104 unit tests (python 3.7 and 3.12; 63 for the baseline, 41 added with the IR models, hybrid ranking, metrics and answer key).

Baseline metadata (setup phase only - baseline NOT implemented, no commit/tag made)

Paper: Statement-Level Software Bug Localization based on Information Retrieval and Spectrum (Jingwen Li et al., 2023)
Defects4J: v1.4.0, commit fcf6b9bb030e0f5f03ebab112eb4d4b622f2f315
buglocalization: bbd67ea65956748cc17a9c26e076aa9933a03008 (Naplues/buglocalization; contains BugLocator)
Java: Temurin OpenJDK 1.8.0_504 (portable, tools/). Python: 3.7.16 (legacy), 3.12.4 (modern). Perl: 5.38.2 (Git msys). OS: Windows 11 native.
Smoke-test bug: Lang-1 (LANG-747). buggy 396afc3e4693cfee182efe582455f2d97058c068, fixed d1a45e9738de5b3e299bb51e987565dcce55fee6 (apache/commons-lang GitHub hashes; d4j's own hashes 2c454a4c.../687b2e62... come from an unavailable mirror, remapped by matching the patch).
Command: bash scripts/run/smoke_lang1.sh
Observed: compiles; NumberUtilsTest 82 run, 1 error = TestLang747 NumberFormatException "80000000", identical to framework/projects/Lang/trigger_tests/1.

Baseline implementation

Code: isbl/baseline/{ir,spectrum,sfl,ranking,pipeline}.py, isbl/baseline/java/CoverageRunner.java, isbl/evaluation/metrics.py. Configs: configs/lang1_k50.json (relevant tests), configs/lang1_k50_alltests.json (full suite).
Outputs: results/raw/Lang-1_tf__k0.5.{json,csv} and ..._sweep_k.csv.
Observed (Lang-1, k=0.5): buggy file IR rank 8/108; statement rank 6 (worst ties) / 3.5 / 1 (best); ISBL equals plain Ochiai/DStar on this bug; MRR 0.167 (worst ties). Details: docs/reproduction.md.
History: three commits (environment and smoke test, baseline code and tests, results and analysis). The tag baseline-authors-v1 from the original plan is not created.

Environment audit (2026-09-30)

Machine: Windows 11 Home 10.0.26200, NVIDIA RTX 2050 4GB, 24GB free on C:. Not admin. Native Windows used; WSL (Ubuntu, WSL2) present but NOT used.
Pre-existing: git 2.49, Python 3.9/3.11/3.12/3.14, conda, Maven 3.9.9, Git-for-Windows perl 5.38.2 (msys). Missing: Java, SVN, wget, Ant.
The three repos the prompt says were cloned did not exist. Created from public sources:
- buglocalization = github.com/Naplues/buglocalization @ bbd67ea6 (2018). Contains BugLocator Java source (+lucene jars), Perl/Python VSM scripts, Eclipse/AspectJ/SWT/JDT/Tomcat/Birt datasets. It contains NO Defects4J / ISBL / Ochiai code. Assumption: this is the intended repo - unconfirmed.
- defects4j = rjust/defects4j tag v1.4.0 @ fcf6b9bb (detached HEAD).
- BugLocator: no separate repo; the copy inside buglocalization/BugLocator is used.
Compatibility problems: (1) d4j 1.4.0 init.sh downloads from people.cs.umass.edu -> 404 (repos archive gone); (2) d4j is Perl+bash+SVN, Perl DBI/DBD::CSV not installed and no SVN, so the d4j CLI was not run; (3) d4j 1.x targets Java 7, javac 8 fails on TypeUtilsTest (generics inference).
Proposed: JDK8 portable + conda py3.7 (legacy) + py3.12 venv (modern). Unresolved: d4j CLI on Windows, SVN, coverage tool, full 6-project repo set, Java 7.

ISBL Reproduction Workspace

Local, Windows-native reproduction of the paper **"Statement-level Software Bug Localization based on Information Retrieval and Spectrum"** (Li, Yue, Lan, Cui; IEEE SMC 2023).

> **Status (2026-10-25): three bugs (Defects4J Lang-1, Lang-2, Lang-3) work end to end. The paper is NOT reproduced.**
> The paper evaluates 373 bugs; this workspace has run 3. Nothing here is claimed to match the paper's numbers.
> Newest work (IR models, extra metrics, hybrid ranking, cosine audit, Lang-2/3): see "14. Improvements and other Lang bugs" at the end and `docs/experiments.md`.
> The upstream clones, JDK and Python environments are not stored in this repository; see "Setting up from a fresh clone" below.

1. What the project is

A bug is hidden somewhere in a large program. The paper's method (ISBL) finds the buggy **line** using two clues:

Clue	Question it answers	How
Bug report (text)	Which **files** look related to the complaint?	Compare the report's words with each source file's words (cosine similarity) and keep the top k% of files.
Tests (pass/fail + which lines ran)	Which **lines** do the failing tests point at?	Record which lines each test runs, then score each line with a formula (**Ochiai** or **DStar**).

ISBL does the file filter first, so only lines inside likely files compete in the scoring. The paper claims this beats plain Ochiai/DStar.

2. Pipeline (what runs, in order)

```
START  scripts/run/demo.ps1  (sets Java 8 + Python 3.7)
  :
  v
Checkout buggy Lang-1, compile, run tests ........ scripts/run/smoke_lang1.sh   -> 1 test fails
  :
  v
INPUT  bug report (data/bug_reports/Lang-1.json) + config (configs/lang1_k50.json, k = 50%)
  :
  v
[A] IR step ..................................... isbl/baseline/ir.py
    cosine(bug report, each of 108 files) -> keep top 50% = 54 files
  :
  v
[B] Coverage .................................... isbl/baseline/java/CoverageRunner.java + JaCoCo
    run tests, record which lines each test executes (and pass/fail)
  :
  v
    Clean: only Defects4J's documented trigger test counts as failing
  :
  +-----------------------------+
  v                              v
FULL spectrum                  REDUCED spectrum (paper section II-B)
(no IR filter)                 drop lines of dropped files, then drop tests covering nothing left
  :                             :
  v                             v
[C] Score every line: Ochiai and DStar ........... isbl/baseline/sfl.py
  :                             :
  +-------------+---------------+
                v
    Rank lines (ties: worst / average / best) .... isbl/baseline/ranking.py
                v
[D] Compare with the real buggy lines ............ isbl/evaluation/metrics.py, diagnostics.py
    (answer key = lines changed by the developers' fix; never used by the search)
```

```
                v
END    results/raw/*.csv, *.json, *_timing.csv  + printed tables
```

An older HTML flowchart is in `docs/flowchart.html` (first version of the pipeline only).

3. The bug: Defects4J Lang-1 (LANG-747)

- **Project:** Apache Commons Lang, method `NumberUtils.createNumber(String)` in `org/apache/commons/lang3/math/NumberUtils.java`.
- **Symptom:** `createNumber("0x80000000")` throws `NumberFormatException: For input string: "80000000"`.
- **Cause:** the code decides between `int` and `long` by counting hex digits. Eight hex digits are treated as `int`, but a value from `0x80000000` to `0xFFFFFFFF` does not fit in an `int`.
- **Real buggy lines (answer key):** lines **467, 468 and 471** of `NumberUtils.java`.
- **Failing test:** `TestLang747` in `NumberUtilsTest`. It matches Defects4J's own recorded failure (`defects4j/framework/projects/Lang/trigger_tests/1`).
- **Versions:** buggy commit `396afc3e4693cfee182efe582455f2d97058c068`, fixed commit `d1a45e9738de5b3e299bb51e987565dcce55fee6` (apache/commons-lang GitHub hashes, matched by patch because Defects4J's own mirror is offline).

4. Results (Lang-1, k = 50%)

All numbers come from `results/raw/`. "Rank" is the best-ranked real buggy line. Ties are reported three ways because the paper does not say how to break them.

4.1 Main result, relevant tests (173 tests)

Method	Tests used	Statements	Rank worst / avg / best	TOP@1 / 3 / 5 (worst ties)	MRR (worst ties)
Ochiai	173	981	6 / 3.5 / 1	0 / 0 / 0	0.1667
ISBL + Ochiai	170	913	6 / 3.5 / 1	0 / 0 / 0	0.1667
DStar	173	981	6 / 3.5 / 1	0 / 0 / 0	0.1667
ISBL + DStar	170	913	6 / 3.5 / 1	0 / 0 / 0	0.1667

With best-case ties the rank is 1, so TOP@1/3/5 = 1/1/1 and MRR = 1.0. With average ties the rank is 3.5.

4.2 Full Lang test suite (2,273 tests after cleaning)

Method	Tests used	Statements	Rank worst / avg / best	MRR (worst ties)
Ochiai	2273	11,102	6 / 3.5 / 1	0.1667
ISBL + Ochiai	1569	6,593	6 / 3.5 / 1	0.1667
DStar	2273	11,102	6 / 3.5 / 1	0.1667
ISBL + DStar	1569	6,593	6 / 3.5 / 1	0.1667

The IR filter removes **40.6%** of statements here, and the buggy line's rank is unchanged.

4.3 What the IR (text matching) step found

Metric	Value
Buggy file	`math/NumberUtils.java`
Buggy file IR rank	**8 of 108**
Smallest k that keeps it	7.4% (any k from 10% to 90% keeps it)
Buggy file cosine / top-1 cosine	0.1198 / 0.1560 (`Validate.java`)
Gap between top-1 and top-2	0.0024
Score entropy	6.65 bits (max for 108 files is about 6.75, so the scores are almost flat)
Files kept at k = 50%	54 of 108

4.4 Why the filter did not help on this bug

Fact	Value
Lines tied at the top score	6 (lines 462, 463, 467, 468, 471, 474)
Of those, real buggy lines	3 (467, 468, 471)
Tests that run those six lines	4: the failing `TestLang747` plus passing `testCreateNumber`, `testCreateNumberMagnitude`, `testIsNumber`
(ef, ep, nf, np) of each tied line	(1, 3, 0, 166)
Ochiai / DStar score of each tied line	0.50 / 0.33
Distinct score values across 913 statements	only 5
Exam score (share of the list read)	0.0061 plain, 0.0066 with ISBL

No test runs one of the six lines without the others, so no spectrum formula can separate them. The IR filter removes unrelated statements but cannot split ties inside the one file that must stay. **On this bug, ISBL equals plain Ochiai/DStar.**

4.5 Effect of k (relevant tests, ISBL)

k	Files kept	Buggy file kept?	Statements	Rank worst / avg / best
10%	11	yes	481	6 / 3.5 / 1
20%	22	yes	671	6 / 3.5 / 1
30%	33	yes	811	6 / 3.5 / 1
40%	44	yes	913	6 / 3.5 / 1
50%	54	yes	913	6 / 3.5 / 1
60% to 80%	65 to 87	yes	913	6 / 3.5 / 1
90%	98	yes	981	6 / 3.5 / 1

The result is the same at every k, because the buggy file always survives and the tie is inside it.

4.6 Where the time goes

Scope	Total	Coverage collection (JVM + JaCoCo)	All other stages
Relevant tests (173)	9.9 s	9.7 s (98%)	about 0.2 s
Full suite (2,280 tests)	231.9 s	231.1 s (99.6%)	under 1 s

Inside the JVM coverage run:

Part	Relevant tests	Full suite
JaCoCo class analysis, repeated after every test	97.7%	93.5%
Running the project's own tests	0.6%	6.0% (13.9 s for 2,280 tests)
JaCoCo dump of coverage data	0.1%	0.1%
JVM startup and other	1.5%	0.4%

IR (about 0.12 s), spectrum building, scoring and metrics are negligible at one-bug scale. A repeated-analysis fix is the obvious first optimisation, but it is not implemented. (A run right after opening the editor can show `01 load_source_files` as slow because the disk cache is cold. That is not a real bottleneck.)

Setting up from a fresh clone (Windows)

Run from the repository root (Git Bash for the `bash`/`mvn` lines).

```
# 1. Pinned upstream sources (ignored by git)
git clone https://github.com/rjust/defects4j.git && git -C defects4j checkout v1.4.0
git clone https://github.com/Naplues/buglocalization.git && git -C buglocalization checkout
bbd67ea65956748cc17a9c26e076aa9933a03008
git clone https://github.com/apache/commons-lang.git work-lang

# 2. Portable Temurin JDK 8 (8u504) unzipped to tools/jdk8u504-b01 (no admin rights needed)
```

```
# 3. JaCoCo 0.8.12 + ASM jars, then compile the per-test coverage runner
source scripts/setup/env.sh
mvn -f scripts/setup/jacoco-pom.xml dependency:copy-dependencies -DoutputDirectory=tools/jacoco
mkdir -p tools/runner && javac -cp "tools/jacoco/*" -d tools/runner isbl/baseline/java/CoverageRunner.java

# 4. Python environments
conda create -p envs/legacy37 python=3.7 -y && envs/legacy37/python -m pip install -r requirements/legacy.txt
pytest==7.4.4
py -3.12 -m venv envs/modern312 && envs/modern312/Scripts/python -m pip install -r requirements/modern.txt -r
requirements/development.txt

# 5. Check out, compile and test Lang-1 (buggy source + fixed-commit tests), then write the test classpath
bash scripts/run/smoke_lang1.sh
(cd work-lang && mvn -q dependency:build-classpath -Dmdep.outputFile=../data/cp.txt)
```

`data/cp.txt` holds absolute paths for this machine and is not committed. The first pipeline run needs `--recollect` to create the coverage file.

5. How to run it

All commands are for PowerShell, from this folder.

```
cd <path-to-this-repository>

# 1. Full demo: versions, checkout, compile, failing test, 104 unit tests, baseline with explanations (about 1
minute)
.\scripts\run\demo.ps1
# if PowerShell blocks scripts:
powershell -ExecutionPolicy Bypass -File .\scripts\run\demo.ps1

# 2. Just the baseline with all the printed tables (Java 8 + Python 3.7 on the PATH first; note the dot-space)
. .\scripts\setup\env.ps1
python -m isbl.baseline.pipeline configs\lang1_k50.json --explain

# 3. Unit tests only
python -m pytest -q tests

# 4. Full Lang test suite (about 4 minutes the first time; add --recollect to re-run coverage)
python -m isbl.baseline.pipeline configs\lang1_k50_alltests.json

# 5. k sweep from 10% to 90%
python scripts\run\sweep_k.py configs\lang1_k50.json

# 6. Excel workbook: one sheet per results CSV, plus a formula-driven Summary and a Guide
envs\modern312\Scripts\python scripts\run\build_excel.py     # writes results\Lang-1_results.xlsx
```

Open `results\Lang-1_results.xlsx` in Excel (formulas calculate when it opens; previewers that do not calculate may show blank summary cells).

Flags for the pipeline: `--explain` prints the cosine table and the coverage matrix. `--recollect` re-runs the tests and JaCoCo instead of reading the saved coverage file.

6. Output files (in `results/raw/`)

File	Contents
`Lang-1_tf_relevant_k0.5.csv` / `.json`	Result row per method, with TOP@N, MRR and all extra metrics
`..._timing.csv`	Seconds and share per pipeline stage, plus the JVM breakdown
`..._ir_cosine.csv`	Cosine similarity of all 108 files
`..._spectrum_scores.csv`	ef, ep, nf, np, Ochiai, DStar and rank for each statement
`..._coverage_matrix.csv`	Full test x statement matrix (open in Excel)
`..._sweep_k.csv`	The k sweep
`Lang-1_tf_all_*`	The same files for the full-suite scope

Metrics logged per method row: TOP@1/3/5/10, MRR, worst/average/best rank, IR buggy-file rank and percentile, cosine, top-1 gap, score entropy, recall@k, matrix density, statements and tests removed, exam score, wasted effort, mean and median buggy rank, number of distinct scores, tie-group size and how many of it are buggy.

7. Repository layout

```
configs/                lang1_k50.json (relevant tests), lang1_k50_alltests.json (full suite)
data/                   bug reports, saved coverage files, classpath (git-ignored)
docs/                   environment-audit.md, baseline.md, reproduction.md, flowchart.html
isbl/baseline/          ir.py, spectrum.py, sfl.py, ranking.py, timing.py, report.py, pipeline.py
isbl/baseline/java/     CoverageRunner.java (per-test JaCoCo coverage + its own timing)
isbl/evaluation/        metrics.py (TOP@N, MRR), diagnostics.py (extra metrics)
requirements/           legacy.txt (py3.7), modern.txt (py3.12), development.txt
results/raw/            all outputs
scripts/setup/          env.ps1, env.sh (put Java 8 + Python 3.7 on the PATH)
scripts/run/            demo.ps1, smoke_lang1.sh, sweep_k.py
tests/                  104 unit tests
tools/, envs/           portable JDK 8, JaCoCo, compiled runner; Python environments
buglocalization/        Naplues/buglocalization @ bbd67ea6 (contains the authors' IR script and BugLocator)
defects4j/              rjust/defects4j, tag v1.4.0 @ fcf6b9bb
work-lang/              scratch checkout of apache/commons-lang (the program being debugged)
```

8. Environment

Item	Version
OS	Windows 11, native (WSL not used)
Java	Temurin OpenJDK 1.8.0_504 (portable, `tools/`)
Python, historical	3.7.16 (`envs/legacy37`), numpy 1.21.6, scikit-learn 1.0.2
Python, modern	3.12.4 (`envs/modern312`), numpy 2.5.3, scikit-learn 1.9.1
Coverage tool	JaCoCo 0.8.12
Defects4J	v1.4.0 (metadata only; the Defects4J command line never ran)
Build	Maven 3.9.9
Perl	5.38.2 (Git for Windows). SVN not installed.

9. Tests

104 unit tests pass on Python 3.7 and 3.12 (`python -m pytest -q tests`). They cover cosine similarity (identical, orthogonal, zero and empty vectors, repeated terms, keywords, camelCase and snake_case), Ochiai and DStar edge cases (all tests fail, all pass, covered by every test, covered by none), spectrum reduction, ranking ties, TOP@N and MRR (rank 1, 3, 5, beyond N, missing and multiple buggy lines), the extra diagnostics, and the timer. One test checks that the IR matches the authors' scikit-learn pipeline to 1e-9.

10. Known differences from the paper (all unresolved)

1. **File filters.** The paper uses BugLocator (rVSM) and Gharibi to filter files. This workspace implements only a plain vector-space version with raw term frequency, Java keyword and stop-word removal. Neither paper tool is implemented.
2. **Identifier splitting** is not done, because the paper does not say. `ir_mode: "tfidf"` (the authors' repo script) is available as an alternative, and it ranks the buggy file 1st instead of 8th.
3. **Tie handling and the DStar exponent** (set to 2) are my assumptions. The paper states neither, and its printed DStar formula is garbled in the PDF.
4. **Coverage tool.** JaCoCo line coverage replaces Defects4J's own coverage tooling. Only main-code lines are counted as statements.
5. **Failing tests.** Only the Defects4J trigger test counts as failing. Seven other failures in the full suite (locale and Java 8 versus 7 differences) were removed, and abstract test base classes are skipped.

6. **Workarounds.** Commit hashes were remapped to GitHub history, Maven replaces Defects4J's Ant, and `TypeUtilsTest.java` was excluded because javac 8 cannot compile it.
7. **Repository assumption.** `buglocalization` is Naplues/buglocalization, chosen from a GitHub name search and not confirmed. It contains no ISBL or Defects4J code, so ISBL here is rebuilt from the paper's text.

11. What is not done

- Only 1 of 373 bugs, in 1 of 6 projects.
- No comparison with the paper's Table III (Ochiai TOP@1/3/5 = 36/84/105, MRR 0.217; ISBL 40/86/109, MRR 0.224) or Table IV (k sweep).
- BugLocator and Gharibi filters.
- The Defects4J command line (needs SVN and Perl modules; its project download host is gone).
- Coverage-speed optimisation.
- No improvements (BM25, TF-IDF comparison, identifier splitting, semantic retrieval, adaptive k).
- The tag `baseline-authors-v1` and branch `research-improvements` from the original plan are not created.
- `docs/architecture.md` is empty, and `docs/experiments.md` does not exist.

12. Glossary

- **IR (information retrieval):** ranking files by how similar their words are to the bug report.
- **Cosine similarity:** a score from 0 to 1 for how alike two word-count vectors are.
- **Spectrum:** the record of which lines each test executes, plus each test's pass/fail result.
- **ef / ep / nf / np:** for one line, the number of failing tests that run it, passing tests that run it, failing tests that miss it, and passing tests that miss it.
- **Ochiai:** `ef / sqrt((ef + nf) * (ef + ep))`. **DStar:** `ef^2 / (ep + nf)`. Higher means more suspicious.
- **k:** the share of top-ranked files kept as suspicious (50% in the paper's recommended setting).
- **TOP@N:** the number of bugs whose real buggy line is in the top N of the ranking. **MRR:** the average of 1/rank.
- **Exam score:** buggy rank divided by the number of statements, the share of the list a developer reads. **Wasted effort:** statements read before the bug.
- **Tie group:** statements with exactly the same score as the buggy line.

13. More detail

`docs/reproduction.md` (step-by-step answers to the plan's 18 questions), `docs/baseline.md` (pinned versions and commands), `docs/environment-audit.md` (what was found and the blockers), `docs/experiments.md` (IR models, metrics, hybrid ranking, Lang-2/3), `docs/literature.md` (papers scanned for metrics and methods), and **`docs/REPORT.md`** (the full illustrated report: original and new pipeline diagrams, every metric with formula and example, Lang-1 and Lang-2 results).

14. Improvements and other Lang bugs (Lang-1, Lang-2, Lang-3)

Requested after faculty review: more comparison metrics, a proper check of the cosine similarity, tf-idf and better IR measures, papers for new metrics, and runs on other Lang bugs. Scope is **three bugs**; every configuration was fixed in advance and nothing was tuned on them, so the numbers are indicative only.

Bug	Type	Buggy lines	IR rank of the buggy file (paper's raw-tf cosine / tf-idf)
Lang-1 (LANG-747)	wrong condition	`NumberUtils.java` 467, 468, 471	8 / 1
Lang-2 (LANG-879)	missing validation (omission)	`LocaleUtils.java` 92	23 / 1
Lang-3 (LANG-693)	missing precision branches (omission)	`NumberUtils.java` 593, 597, 601, 605	16 / 1

What the experiments show (details, tables and caveats in `docs/experiments.md`):
- **Cosine audit:** our cosine equals scikit-learn and SciPy to 1e-16 and has the expected properties. The weakness is the raw term-frequency *weighting*: the wrong top-ranked file gets 88-100% of its score from words found in most files.
- **IR models:** idf-weighted models (tf-idf, rVSM, LSI) rank the buggy file 1st on all three bugs; the paper's raw-tf cosine ranks it 8th / 23rd / 16th. Stripping the pasted stack trace in Lang-2's report (49 foreign frames) helps most models.
- **False exclusion:** at k = 10% the paper's raw-tf IR throws the buggy file away on 2 of 3 bugs; tf-idf, rVSM and a BLUiR-style model keep it while cutting 44-47% of the statements.
- **SFL formulas:** 11 formulas give the same ranking (except Wong2, far worse): statements with identical (ef, ep, nf, np) cannot be separated by any formula.
- **Hybrid ranking:** breaking exact SFL ties with the bug-report words on each statement lowers the expected rank from 3.5 to 2 on Lang-1 and Lang-3 (no change on Lang-2); a plain linear blend is worse than Ochiai. This is our exploratory extension, picked post hoc among 12 variants on the same three bugs.

```
. .\scripts\setup\env.ps1
python scripts\bugs\prepare_lang_bug.py 2 3        # prepare Lang-2, Lang-3 (Maven + internet + JDK 8; one time)
python scripts\run\cosine_audit.py                 # cosine correctness and per-term diagnosis
python scripts\run\run_experiments.py              # IR models x SFL formulas x ISBL x hybrids, about 50 s
envs\modern312\Scripts\python scripts\run\build_excel_experiments.py    # results\Lang-1-2-3_experiments.xlsx
.\scripts\run\demo_experiments.ps1                 # the two commands above that print tables
python -m isbl.baseline.pipeline configs\bugs\Lang-2.json --explain     # standard pipeline on another bug
```

New code: `isbl/baseline/ir_models.py` (12 IR models + options), `hybrid.py`, `answer_key.py`, extra formulas in `sfl.py`, extra metrics in `isbl/evaluation/metrics.py`, `isbl/experiments/engine.py`, `scripts/bugs/prepare_lang_bug.py`. The coverage runner now analyses only executed classes (about 8x faster, identical coverage). Config key `ir_model` selects any model in the standard pipeline (e.g. `configs/lang1_tfidf_sublinear.json`).